Dust Particles and Gas+Dust Self-Gravity in the AthenaK Shearing Box

Implementation Document

Phase 4a: the dust-module merge; Phase 4b (i): the shear-periodic deposit fold; (ii): the ideal-gas energy update; (iii): the particle timestep under shear; Phase 4c: dust self-gravity; Phase 4d: the vertical policy, restart, diagnostics; Phase 5: particles on statically refined meshes; gravity on statically refined meshes; Phase 6: particles on adaptively refined meshes, and the image-free finest-kernel deposits

commits `449b8f09` (merge), `a7c0def3`, `774c6519`, `78320aee` (second merge), `6c0cafef`, `489c656c`, `375643e6`, `9a96f9f3`, `d`

September 3–15, 2026; image-free deposits October 19, 2026

Abstract

This document describes *how* Lagrangian dust particles are being brought into the `athenak-multigrid` tree — the shearing-box code with multigrid self-gravity on refined meshes documented in `multigrid_shear_implementation.pdf` — on the way to gas+dust self-gravity in a stratified, cooling, gravito-turbulent box (the configuration of Baehr, Zhu & Yang 2022, with the radial pressure gradient they omitted). It is the companion to `dust_selfgravity_validation.pdf`, which records what the resulting code does on test problems. Both documents are meant to grow phase by phase.

Phase 4a, recorded here, merges the sibling fork `athenak-dust` (particle-mesh dust with IMEX drag, validated in its own repository) into the multigrid tree. The merge itself is mechanical — eight keep-both conflicts and one renumbered output slot — but it exposed a real defect: the dust module's additive ghost-deposit exchange was written against the per-neighbor MPI messaging of July's upstream, while this tree carries the rank-packed messaging of upstream #775. Any multi-rank dust run deadlocked at cycle 0. The exchange now uses the aggregated-message path with the same idiom as the cell-centered exchange, and ranks 1, 2 and 4 agree to every printed digit. A second follow-up aligns the dust task graph with the tree's Prolongate-before-boundary-conditions order.

A **second merge** the same day caught the tree up with the dust fork's own latest work (seven commits, to `1f3e1ed9`): the fork had independently made the identical rank-packed rewrite of the deposit exchange, so that conflict resolves to their version; and it now offers two *predictor–corrector* coupling modes (`coupling = pc2` and the adaptive `hybrid`) that run under plain `rk2`, second order in the resolved drag regime with a stiff backward-Euler fallback. Dust can therefore run under the same integrator as every gravity validation before it; the `imex2+` path remains for stiff cases.

Phase 4b (i) closes the first of the gaps the survey named: ghost deposits of the MeshBlocks on the shear-periodic x_1 faces are now folded across the faces with a conservative azimuthal remap — gathered into global y–z planes, shifted by the shear offset with the Phase-1 remap-flux kernels, and *added* into the active edge strips of the blocks across the face — while the unsheared plain-periodic x_1 contributions of those blocks are suppressed. The fold is the adjoint of the copy-exchange ghost fill, so its shift signs are the reverse of the multigrid fill's. A unit-test problem generator certifies it exactly at integer shifts and at second/third order otherwise, and a structured 3D run shows the old unsheared exchange was a percent-level error. **Phase 4b**

(ii) lifts the isothermal restriction on back-reaction: every drag kick on the gas momentum now goes through one helper that, for an ideal gas, books the matching kinetic-energy change so the internal energy is untouched, and a flagged option deposits the frictional dissipation of each particle kick as heat. The residual energy errors are the convexity gap of the low-storage RK combination, first order in the timestep, and are quantified. **Phase 4b (iii)** removes the last of the three Phase-4b gaps: the particle transport timestep no longer resolves the background shear $-q\Omega_0 x$ cell by cell (a $1/L_x$ penalty the orbitally advected gas does not pay) but limits the peculiar velocity per cell and the *total* transport velocity per MeshBlock — the latter being what the single-hop particle migration and the one-layer deposit halo actually require — with the bound made exact for the low-storage tableaus through their largest stage fraction. In an $L_x = 8$ box the step grows 5.3× to the gas limit. **Phase 4c** adds the dust to the Poisson equation and the potential to the particles. The particle-mesh dust density is deposited before each stage's solve, exchanged additively (with the shear fold) and then copied into its ghost layer, and registered with the gravity module as an *extra density* that both solvers add to the gas density at their single source read; after the solve the force $-\nabla\phi$ is formed on the mesh, exchanged like the module's provisional gas velocity and gathered onto the particles with the deposit kernel. Because scatter and gather share one kernel and the gradient is the adjoint of the symmetric Laplacian, the coupling conserves the total gas+dust momentum to round-off by construction. Two limitations surfaced on the way and are recorded: the multigrid needs cubic cells and more than one MeshBlock across each periodic direction, and 3D runs on cubic cells grow a grid-scale mode once $c_s\Delta t/\Delta z \gtrsim 0.35$. **Phase 4d** closes the dust track's remaining gaps. The instability just named is identified — by its eigenmode and by a seeded gas-only run — as the shearing-box hydro scheme's own 3D checkerboard mode at Courant numbers above ~ 0.35, not a property of the dust coupling, which merely seeded it; the rule `cfl` ≤ 0.3 in 3D is a hydro rule the gravito-turbulence runs already follow. Particles crossing a physical vertical face are removed with their mass accounted (the Athena-C `zbc_out = 1` policy); particle arrays are written to and read from restart files; the module's own particle-mesh density is an output variable; and the SC14 history gains the dust's mass, momenta, kinetic energy, Reynolds stress and escaped mass. **Phase 5** takes the particles onto statically refined meshes. Its central finding is that the natural conservative exchange of ghost deposits across a level jump (restrict fine, inject coarse) leaves an $O(1)$ two-cell ripple in the deposited density at every interface — a particle's kernel reaches half as far on the fine side — and that no local rule is both conservative per particle and consistent. The scheme adopted deposits with the finest-level kernel everywhere (blocks one level below it through a 2× fine image averaged onto their cells), which is conservative, consistent, and exact for a uniform lattice at the finest spacing; the additive exchange, the prolongation of the module's copy-exchanged fields, a defect in upstream's particle routing to coarser neighbors and the shear machinery at the face level complete it. **Phase 6** takes the particles onto adaptively refined meshes: they are routed to their new MeshBlocks inside the remesh (across ranks through the ordinary particle pipeline), the dust module rebuilds its per-block state, a criterion on the particle-mesh dust density drives the refinement, and the particles of a refining block are split into 2^d half-fine-cell children so a population keeps its sampling per cell on the finer level — the answer to the aliasing the linA mode exposed in Phase 5. The 2D r–z box, whose shear machinery is inert, is freed from the ring and face policies and refines in patches; the Johansen & Youdin (2007) AB and BA runs are set up on it. The fine images themselves were later retired (§12.6): a block r times coarser than the finest level now folds the finest-level kernel onto its own cells directly, and only a block with a finer neighbour keeps a fixed 2× halo image for the exchange, so the deposit memory no longer grows by 8 per level.

The document also records, for the phases that follow, the integration map derived from the code survey: where the gravity solve is called relative to the dust task graph, the three places the solver reads density, the ghost-width contract of the potential, and the design chosen for the particle force (grid gradient, exchanged and shear-remapped like the module's provisional gas velocity, gathered with the deposit kernel), together with the known gaps that Phases 4b–4d must close.

Contents

1 Scope, sources, and how to read this **5**
- 1.1 What this document is . . . 5
- 1.2 The program . . . 5
- 1.3 Commit provenance . . . 5
- 1.4 Files touched . . . 6

2 The target problem and the design it implies **6**
- 2.1 What is being built . . . 6
- 2.2 The integration map . . . 6

3 The dust module as inherited **7**
- 3.1 Data layout . . . 8
- 3.2 The particle-mesh kernel . . . 8
- 3.3 The stage algorithm . . . 8
- 3.4 The task graph . . . 8
- 3.5 Shearing-box conventions . . . 9
- 3.6 The particle timestep . . . 9
- 3.7 Guards and parameters . . . 9

4 Phase 4a: the merge **9**
- 4.1 Route . . . 9
- 4.2 Conflicts . . . 10
- 4.3 Follow-up 1: task order . . . 10
- 4.4 Follow-up 2: the deposit exchange on the rank-packed path . . . 10
- 4.5 The second merge: catching up with the dust fork . . . 11
- 4.6 Build notes . . . 12

5 Phase 4b (i): the shear-periodic deposit fold **12**
- 5.1 The problem . . . 12
- 5.2 The sign . . . 13
- 5.3 The fold . . . 13
- 5.4 Suppressing the unsheared contribution . . . 13
- 5.5 Hooks and parameters . . . 14
- 5.6 The test generator . . . 14
- 5.7 A structured, rank-independent initial condition . . . 14

6 Phase 4b (ii): the ideal-gas energy update **15**
- 6.1 The problem . . . 15
- 6.2 The rule . . . 15
- 6.3 What the bookkeeping cannot do, and the lesson from the first test . . . 15

7 Phase 4b (iii): the particle timestep under shear **16**
- 7.1 The problem . . . 16
- 7.2 What the limit is actually for . . . 16
- 7.3 The rule . . . 16
- 7.4 Parameters and diagnostics . . . 17
- 7.5 What the larger step exposes . . . 17

7.6 What does not change . . . 17

8 Phase 4c: dust self-gravity 17
8.1 Design, as built . . . 17
8.2 Momentum conservation, exact by construction . . . 18
8.3 Synchronous entry points . . . 19
8.4 Parameters, guards, boundaries . . . 19
8.5 What was found on the way . . . 19

9 Phase 4d: the instability identified, the vertical policy, restart, diagnostics 20
9.1 The instability of §8 is the hydro scheme's . . . 20
9.2 Vertical policy: outflow removal . . . 20
9.3 Particle restart . . . 20
9.4 Diagnostics . . . 21
9.5 The dusty Jeans test . . . 21
9.6 The science setup: Baehr, Zhu & Yang (2022) in two stages . . . 22

10 Phase 5: particles on statically refined meshes 24
10.1 What the dust module relies on, and what refinement breaks . . . 24
10.2 The deposit kernel across a level jump . . . 24
10.3 The additive exchange across levels . . . 25
10.4 Copy exchanges, prolongation, and the shear machinery . . . 26
10.5 Particle migration . . . 26
10.6 Files, hooks, parameters . . . 26

11 Gravity on statically refined meshes 26
11.1 `MeshRefinement` now precedes the `ProblemGenerator` . . . 27
11.2 Volume weighting of the `fft_poisson` diagnostics . . . 27
11.3 New diagnostics . . . 27
11.4 The coarse-fine boundary layer of the potential . . . 28
11.5 Files, inputs, parameters . . . 28

12 Phase 6: particles on adaptively refined meshes 29
12.1 Redistribution inside the remesh . . . 29
12.2 Splitting on refinement . . . 29
12.3 The dust density criterion . . . 30
12.4 The 2D r–z shearing box . . . 30
12.5 Any number of levels . . . 31
12.6 Image-free deposits . . . 31
12.7 Files, hooks, parameters . . . 32

13 What the next phases must add 33

14 Input-file interface 34

1 Scope, sources, and how to read this

1.1 What this document is

This is an implementation reference for the dust track. For each phase it answers: what the code relies on, where that is established and enforced, which data structure carries the new state, which function does the work, and from which call site it is reached. Results live in the companion documents:

`validation/dust_selfgravity_validation.pdf`	results, tests, figures (this track)
`validation/multigrid_shear_implementation.pdf`	the gravity/refinement machinery being extended
`validation/multigrid_selfgravity_validation.pdf`	its validation record (Phases 0–3)
`../athenak_dust_tests/`	the dust module's own validation notebooks and inputs

1.2 The program

The target science is the interaction of gravitational instability of the gas with dust concentration and collapse in a shearing box: Baehr, Zhu & Yang (2022, ApJ 933, 100) in 3D stratified, cooling, gravito-turbulent boxes with superparticles at $\mathrm{St} = 0.1$–10, plus the radial pressure gradient (and hence the streaming instability) that their setup left out, and eventually adaptive zoom on the dust clumps. The phases:

Phase	Status	Content
4a	**done**	merge `athenak-dust` into the multigrid tree; establish that both the dust module and the gravity machinery are unchanged by the merge
4b	(i), (ii) **done**	(i) shear-periodic *additive* deposit remap (§5); (ii) ideal-gas energy update under drag back-reaction (§6); (iii) particle timestep on the shear-subtracted velocity; adopt the PC2 coupling (§4.5) as the production path
4c	planned	dust self-gravity: mass deposit into the Poisson source, three density reads, force gather to particles; dust-only and gas+dust tests
4d	planned	the science configuration: SC14 box + pressure gradient + dust; particle restart; clump diagnostics
5	**done** (static)	particles on statically refined meshes: finest-level deposits, the additive exchange across levels, prolongated copy exchanges, migration to coarser neighbors (§10); adaptive remesh deferred

1.3 Commit provenance

The merged fork is `github.com/athenak-dust/athenak`, branch `dust`. At the first merge it stood at `d6de1296`, thirteen commits on top of upstream AthenaK as of July 8, 2026 (commit `5f199310`, the merge base with this tree). The one that matters is `e785735e` "Add IMEX dust-particle coupling to hydrodynamics" (36 files, +3864 lines); the rest are the matrix-free coupled drag solvers, GPU migration fixes, a snapshot-seeded clump benchmark, and diagnostics. The local clone turned out

to be two weeks stale; the second merge (§4.5) brought in the seven commits through `1f3e1ed9` (September 2). The remote is registered locally as `dust` (`git remote add dust ../athenak-dust`), so later fixes in that fork come in with a plain `git merge dust/dust` — which is exactly what the second merge was.

1.4 Files touched

File	Role
`src/dust/dust.{cpp,hpp}`	`DustGasDrag` module: parameters, arrays, boundary objects, guards, the shared PM kernel `dust::PMWeights`
`src/dust/dust_tasks.cpp`	the combined hydro+particle task graph (`AssembleDustGasDragTasks`) and the boundary tasks
`src/dust/dust_pm.cpp`	deposit, per-cell implicit solve, gather/kick, back-reaction
`src/dust/dust_push.cpp`	explicit particle push (Coriolis, tide, drift, drag history)
`src/dust/dust_newdt.cpp`	particle transport timestep, stopping-time bookkeeping, the γ-switch
`src/dust/dust_solver.cpp`	optional matrix-free coupled drag solvers
`src/bvals/bvals_dep.cpp`	`MeshBoundaryValuesDep`: additive ghost-deposit exchange (rewired in 4a)
`src/bvals/bvals_part.cpp`, `bvals.{cpp,hpp}`	shear-periodic particle routing; deposit-exchange class declaration
`src/driver/driver.{cpp,hpp}`	`SetImEx2PlusCoefficients` (the γ-switch entry point)
`src/mesh/meshblock_pack.{cpp,hpp}`	`pdust` registration; hydro/particle task lists ceded to the dust module when a `<dust>` block exists
`src/mesh/mesh.{cpp,hpp}`, `build_tree.cpp`	`<time>/dtmax` hard cap
`src/athena.hpp`	widened `ParticlesIndex` (17 real, 3 integer fields)
`src/outputs/*`	`dust_d` derived output; particle velocities in `pvtk`; slot renumbering
`src/pgen/tests/dust_*.cpp`, `streaming_linear.cpp`	the four built-in dust problem generators
`inputs/dust/*.athinput`	their reference inputs

2 The target problem and the design it implies

2.1 What is being built

The reference implementation for particle self-gravity in a shearing box is the Athena-C problem generator `parsg_strat3d.c` in `athena_parsg/src/prob/` (Rixin Li's fork). Its method, with `PLUS_GAS_SELF_GRAVITY`: deposit the dust mass onto the grid with the TSC kernel, *add the gas density into the same buffer*, solve *one* Poisson equation, difference Φ on the grid, and interpolate the resulting acceleration to the particles with the *same* TSC kernel used for the deposit. The gas feels the total potential as a body force. Baehr et al. do the same in Pencil (their eq. 15, $\rho = \rho_g + \rho_d$ in a periodic FFT). Two details of the Athena-C version are not to be copied: its deposit omits the cell-volume division and hides it in the particle mass, and its drag feedback must subtract the gravitational kick because gravity is folded into the same implicit particle update as drag.

2.2 The integration map

The survey of this tree fixed where each piece attaches. These are facts about the code as merged; the items marked *planned* are the Phase 4b/4c designs.

Where gravity is solved. Gravity has no task-list entry. The driver calls `Gravity::Solve(stage)` once per explicit stage, *between* the `before_stagen` and `stagen` task lists (`driver/driver.cpp`, lines 563–569). Anything the solver must see — the dust mass deposit and its ghost exchange — has to complete inside `before_stagen` (*planned*: a deposit task there, on the stage-start particle positions).

The three density reads. The solver reads gas density in three places, and all three must learn about dust (*planned*, Phase 4c):

- `gravity/mg_gravity.cpp:353`, `LoadSource(u0, IDN, ...)` into the multigrid source: loads active zones *plus one ghost layer*, so the dust deposit must already be additively ghost-exchanged one layer deep;
- `multigrid/multigrid_slab.cpp:263`, the slab-plane gather that builds the open-z Dirichlet planes (active zones, reduced across ranks) — omitting dust here would make the planes inconsistent with the interior source;
- `gravity/fft_gravity.cpp:218`, the FFT oracle's density gather.

The potential's ghost contract. `phi` is allocated with the hydro ghost width but only *one* ghost layer is ever filled (`mg_nghost = 1`; larger values are fatal with shear). The gas source term (`SourceTerms::SelfGravity`, `srcterms/srcterms.cpp:298`) needs exactly that: a centered $-\rho\nabla\Phi$ for momentum and the flux-based energy term of Mullen, Hanawa & Gammie (2020). A TSC gather at a particle in the last active cell reaches one ghost cell of the *acceleration*, i.e. two of the potential. *Planned*: compute $\mathbf{g} = -\nabla\Phi$ on active cells, then copy-exchange and shear-remap the three-component field exactly as the module already does for its provisional gas velocity u^* (`MeshBoundaryValuesCC` plus a `ShearingBoxCC`), and gather with `dust::PMWeights`. Using the deposit kernel for the force gather — not the derivative of the kernel — is the Hockney–Eastwood choice that suppresses self-force, and it is what the Athena-C reference does.

Where the particle force goes. `DustGasDrag::ExplicitPush` (`dust/dust_push.cpp:65`) already assembles the Coriolis and vertical-tide accelerations per particle; the gathered $\mathbf{g}$ joins them there. The back-reaction deposits only the drag kick $\Delta v_j = c_j(\tilde{u}_j^* - v_j)$, so gravity is never double-counted — there is no analogue of the Athena-C `feedback_corrector` subtraction to write.

The gas force. Nothing to add: the existing source term applies whatever potential the solver produced.

Stage structure. Under `coupling = imex`, `imex2+` has three explicit stages of which the third is dormant for explicit physics ($\beta_3 = 0$); the gravity solve should be skipped there or a third of the solver cost is wasted (*planned*). Under the PC2/hybrid couplings of §4.5 the integrator is `rk2` and the gravity solve runs exactly as it does for gas alone.

3 The dust module as inherited

This section is the working summary of `athenak-dust`, with the facts the later phases build on. The design is Yang & Johansen (2016) particle-mesh coupling (one shared kernel for scatter and gather, exact momentum-conserving back-reaction) with the Benítez-Llambay, Krapp & Pessah (2019) closed-form per-cell implicit drag inside the Krapp et al. (2024) IMEX(4,3,2) integrator (`imex2+`), generalized to per-particle stopping times.

3.1 Data layout

Particles are two flat device arrays indexed `(field, particle)`. The field enumeration (`athena.hpp:76`) is

```
enum ParticlesIndex {PGID=0, PTAG=1, PSP=2,
                     IPX=0, IPVX=1, IPY=2, IPVY=3, IPZ=4, IPVZ=5,
                     IPX1=6, IPVX1=7, IPY1=8, IPVY1=9, IPZ1=10, IPVZ1=11,
                     IPRX=12, IPRY=13, IPRZ=14, IPTS=15, IPM=16};
```

so a dust particle carries position, velocity (all three components even in 2D r–z), the low-storage stage-1 registers, the recorded drag rate R_j, its stopping time and its **mass**. Integer data: owning MeshBlock, global tag, species. `nrdata = 17`, `nidata = 3`.

3.2 The particle-mesh kernel

`dust::PMWeights` (`dust/dust.hpp:78`) returns the three weights of a NGP / CIC / TSC stencil in one direction; every deposit, gather, matrix-free operator and kick uses it, which is what makes scatter and gather weights identical by construction. `<dust>/deposit` selects the kernel (default `tsc`); the stencil is always three wide (NGP zeroes the wings). Particle-mesh assignment needs `nghost >= 2`.

3.3 The stage algorithm

Per explicit stage, on the conserved gas state (primitives are stale mid-stage), with $a\,\Delta t = a_{\rm impl}\Delta t$ the implicit weight of the tableau:

1. `DepositDrag`: scatter $Q = \sum_j \mu_j W_{jk}$ and $\mathbf{P} = \sum_j \mu_j W_{jk}\mathbf{v}_j$ with $c_j = a\Delta t/(t_{s,j} + a\Delta t)$, $\mu_j = m_j c_j/V$ (a *drag-weighted* density, not a mass density); additive halo exchange.
2. `GasImplicitSolve`: per cell, $\mathbf{u}^* = (\rho\mathbf{u} + \mathbf{P})/(\rho + Q)$, the closed-form momentum-conserving backward-Euler pair. With `back_reaction = false` the deposits are excluded so test particles are kicked toward the unmodified gas.
3. Copy exchange of $\mathbf{u}^*$ ghosts (plus shear remap in a 3D shearing box).
4. `GatherKickPMBR`: gather $\tilde{\mathbf{u}}^*_j$, kick $\Delta\mathbf{v}_j = c_j(\tilde{\mathbf{u}}^*_j - \mathbf{v}_j)$, record $R_j = \Delta\mathbf{v}_j/(a\Delta t)$, and deposit $-m_j W_{jk}\Delta\mathbf{v}_j/V$ with the *same* weights and the same $\Delta\mathbf{v}$ — exact momentum conservation to round-off.
5. Additive halo exchange of that deposit; `ApplyPMBR` adds it to the gas momentum and rescales it in place into the gas drag rate R_g.
6. Next stage, the history terms $\tilde{a}_{22}\Delta t\, R$ enter on both sides (`AddDragHistoryGas`; the particle side inside `ExplicitPush`).

There is *no* drag timestep constraint for any t_s or dust-to-gas ratio; only particle transport limits Δt (§3.6).

3.4 The task graph

When a `<dust>` block exists, `MeshBlockPack::AddPhysics` skips both `Hydro::AssembleHydroTasks` and `Particles::AssembleTasks` and hands the whole graph to `DustGasDrag::AssembleDustGasDragTasks` (`mesh/meshblock_pack.cpp:265-271`, `dust/dust_tasks.cpp:37-108`), the same pattern as `IonNeutral`. The consequence for later phases:

Ownership. The dust module owns the entire per-stage task graph. Any task the gravity work needs (deposit, exchange, force gather) is inserted into `AssembleDustGasDragTasks`, and the hydro chain inside it must track changes made to `Hydro::AssembleHydroTasks` in this tree (the 4a follow-up `a7c0def3` is the first instance).

The `stagen` sequence: `FirstTwoImpRK` (replaces `CopyCons`) → gas fluxes / update / source terms → `AddDragHistoryGas`; in parallel `ExplicitPush` → particle migration (`NewGID`, counts, sends, receives — *inside every stage*, not once per cycle) → the deposit chain of §3.3 → the gas tail (orbital advection, restrict, exchange, shear exchange, `Prolongate`, `ApplyPhysicalBCs`, `ConToPrim`, timesteps). Every dust task returns early on the dormant third stage.

3.5 Shearing-box conventions

Particle velocities are *shear-subtracted*: the explicit push applies the FARGO-form Coriolis terms $f_x = 2\Omega_0 v_y$, $f_y = -(2-q)\Omega_0 v_x$ and, with `<shearing_box>/stratified`, $f_z = -\Omega_0^2 z$ (`dust/dust_push.cpp:98-100`); positions drift with the *full* azimuthal velocity, $\dot{y} = v_y - q\Omega_0 x$ (`:113-114`). This matches the gas with `orbital_advection = true` (the tree's default and the SC14 setting). Crossing a radial face is therefore a *position-only* transform: wrap x, shift y by $\mp y_{\rm sh}$, $y_{\rm sh} = q\Omega_0 L_x t \bmod L_y$ (`bvals/bvals_part.cpp:76,134`), with the RK position registers shifted by the same amounts so the low-storage combination stays in one periodic image. The destination MeshBlock is found from a precomputed $(y, z) \to$ (gid, rank) map of the face blocks, not from the x_1 neighbor.

The radial pressure gradient is a gas-only constant acceleration through the stock `<hydro_srcterms>/const_accel` machinery ($2\Omega_0\eta v_K$ along x); particles never feel it.

3.6 The particle timestep

`DustGasDrag::NewTimeStep` (`dust/dust_newdt.cpp:95`) takes $\min_p \min_i \Delta x_i / |v_{{\rm transport},i}|$ times `<dust>/dt_cfl` (default 0.5), where in a 3D shearing box $v_{{\rm transport},y} = v_y - q\Omega_0 x$ (`:120`). In a wide box this is the same penalty the gas pays without orbital advection ($\sim 5\times$ at SC14 size). Phase 4b (iii) relaxes it (§7).

3.7 Guards and parameters

Fatal conditions in `DustGasDrag::DustGasDrag` (`dust/dust.cpp`): MHD present (`:45`); 1D mesh (`:50`); an integrator other than `imex2+` (`:63`); `multilevel` (`:69`); boundaries neither strictly periodic nor 3D with shear-periodic x_1 (`:78`); `nghost < 2` (`:93`); back-reaction with a non-isothermal EOS (`:170`); a coupled solver other than `local` with shear (`:227`). A *warning* at `:84` states that ghost deposits are exchanged with the plain-periodic pattern under shear, exact only for azimuthally uniform states. The same `multilevel` guard is repeated in the `MeshBoundaryValuesDep` constructor and in the shear-periodic particle constructor. The parameter interface is reproduced in §14.

4 Phase 4a: the merge

4.1 Route

The two forks share upstream history through `5f199310` (July 8, 2026); the dust fork adds 13 commits, this tree about 140 including an upstream merge on August 10 that brought in #775 (rank-packed boundary messaging) and #612 (`imex2+`). A merge was preferred over cherry-picks: it keeps the dust fork's history and authorship, resolves conflicts once, and makes later fixes from that

fork a one-line pull. The whole sequence was first rehearsed in a scratch clone (conflicts, follow-ups, both builds, the full acceptance battery) and then repeated in the working tree with the rehearsed resolutions copied in.

4.2 Conflicts

Eighteen files were touched by both forks; eight conflicted, all of the same kind — both sides appended adjacent code at the same anchor — and all were resolved by keeping both sides:

File	this tree had added	the dust fork had added
`driver/driver.cpp`	the super-time-stepping controller functions	`SetImEx2PlusCoefficients` (the constructor's inline `imex2+` tableau moved into a function the γ-switch can call)
`mesh/mesh.cpp`, `mesh.hpp`	STS timestep cap and members	`dtmax` hard cap (default ∞, inert)
`mesh/meshblock_pack.cpp`, `.hpp`	physics (9) gravity, `pgrav`	physics (10) dust, `pdust`
`outputs/outputs.hpp`, `basetype_output.cpp`	`grav_phi` at slot 153 and its guard	`dust_d` at slot 153 and its guard
`parameter_input.cpp`	`gravity` in the block-name list	`dust`

The output slot. Output variable names live in one static array, `var_choice`, and the index of the matching name (`ivar`) is used only by the sanity guards that test numeric ranges ("151–153 are particle outputs, so a `<particles>` block must exist"). Each fork had appended one name at position 153 and written a guard for it. Resolution: `dust_d` keeps 153, `grav_phi` becomes 154, `NOUTPUT_CHOICES` 154 → 155, the gravity guard tests 154. Everything downstream selects by name string, so the number appears nowhere else. The 31 auto-merged files were verified identical to the rehearsal.

4.3 Follow-up 1: task order

Upstream reordered the hydro chain to `Prolongate` → `ApplyPhysicalBCs` → `ConToPrim` (so physical boundaries act on prolongated ghosts); the dust graph still had the July order. Commit `a7c0def3` aligns `dust/dust_tasks.cpp:96-98`. Inert on uniform meshes (dust is fatal on refined ones), but the invariant of §3.4 says keep them in lockstep.

4.4 Follow-up 2: the deposit exchange on the rank-packed path

Symptom. The first multi-rank test of the merged tree (the damping problem with drifting particles, `v0 = 0.5`, 4 ranks) hung at cycle 0 with all four processes spinning; the 1-rank run of the same binary reproduced the reference number exactly, and multigrid tests at 4 ranks were bit-identical to serial.

Mechanism. `MeshBoundaryValuesDep` (`bvals/bvals_dep.cpp`) was written as a copy of July's `MeshBoundaryValuesCC`: per-neighbor `MPI_Isend` on `sendbuf[n].vars_req[m]` with `CreateBvals_MPI_Tag(lid, dn)` tags, and per-neighbor `MPI_Test` on `recvbuf[n].vars_req[m]`, relying on the base class `InitRecv` to post the matching per-neighbor receives. In this tree `MeshBoundaryValues::InitRecv` (`bvals/bvals_tasks.cpp:32`) is *rank-packed* since upstream #775: it builds, once per mesh sequence,

a metadata table of all (block, neighbor) payloads per peer rank (`BuildRankPackedVarMetadata`, `bvals/bvals.cpp:134`, which also does a one-shot header exchange so receivers know the pack order), and posts *one* `MPI_Irecv` per peer rank with tag 1 into `rank_recvbuf_vars_`. `ClearRecv`/`ClearSend` wait on those aggregated requests. So the deposit's per-neighbor sends had no receiver, its tests polled requests that were never posted, and the stage never completed.

Fix. Commit `774c6519` re-expresses the two transport steps in the same idiom as the cell-centered exchange (`PackAndSendCC` and `RecvAndUnpackCC` of `MeshBoundaryValuesCC`):

- `PackAndSendDeposit` (`:122`): on-rank neighbors are still written straight into the destination's `recvbuf[dn].vars`; off-rank payloads go into `rank_sendbuf_vars_` at `send_agg_offset_-(m*nnghbr+n)`; after one fence, one `MPI_Isend` per message in `send_var_msgs_` (tag 1, `send_-var_reqs_`);
- `RecvAndSumDeposit` (`:228`): `MPI_Test` over `recv_var_reqs_`; in the (deliberately scalar) loop over neighbors the addend is `aggrbuf(base + bi)` when `recv_agg_offset_(m*nnghbr+n) >= 0` and `rbuf[n].vars(m, bi)` otherwise.

The metadata builder sizes each entry from the buffer index ranges the derived class defines (`GetVarDataSize` → `isame_ndat`), so the reversed data flow of the additive exchange (pack ghosts, sum into active strips) needs no change to the base class. The serial path is unchanged (bit-identical damping ladder); ranks 1, 2 and 4 now agree to every printed digit.

Messaging. Every new boundary-exchange class in this tree must use the rank-packed vars path (`send_var_msgs_` / `recv_var_reqs_` / `*_agg_offset_`); the per-neighbor `vars_req` arrays are dead for vars traffic and only the flux-correction path still posts per-neighbor requests.

4.5 The second merge: catching up with the dust fork

What came in. A question about a colleague's commit revealed that the local dust clone was two weeks behind. `origin/dust` at `1f3e1ed9` adds, oldest first: a CPU particle-mesh optimization (plain addition instead of atomics under `Kokkos::Serial`, weights evaluated in cell coordinates); the NSH generator without dust; *their own* import of upstream #775 (`0d7688b6`), extended to the additive deposit exchange — the same rewrite as `774c6519`, validated by them at 2, 4 and 8 ranks and on CUDA; adaptive PC2 / split-BE coupling (`4b44ba09`); and direct PC2 (`7fc41cd8`).

Conflicts. Three files. `bvals/bvals_dep.cpp`: both sides rewrote the same functions the same way; the fork's version was taken (it additionally rebuilds the rank-packed metadata inside `PackAndSendDeposit` when the variable count or mesh sequence changed, instead of relying on `InitRecv` having run). `bvals/bvals_fc.cpp`: ours kept — the early return on inactive face-centered faces is upstream #739's $\nabla\cdot B$ fix, which the fork's #775 import predates. `mesh/mesh_refinement.cpp`: ours kept — the fork calls `InitBoundaryValuesAndPrimitives` at the ancestor's position, while the Phase-0c code here calls it later, after the shear-state rebuild; keeping both would call it twice. The base boundary-value files auto-merged with *no net change*: the fork's #775 import is textually the upstream copy this tree already had.

The PC2 and hybrid couplings. `<dust>/coupling` now selects `imex` (the default, §3.3, requires `imex2+`), `hybrid`, or `pc2` (both require `rk2`; `gamma_switch` is ignored with a warning). From the fork's own description: the *PC2* branch advances the particles once per cycle without a coupled gas

solve — a fresh old-state drag-feedback predictor is deposited during RK stage 1, the midpoint gas velocity is built after the completed stage-1 boundary tail, the particles take an implicit-midpoint drag update at the predicted midpoint position, the actual drag impulse is deposited at that midpoint, and the final gas momentum is corrected by removing the tableau-weighted predictor. Particles migrate once, after the update. The *split-BE* branch first completes a dust-free hydro RK2 step and then performs one full-Δt coupled backward-Euler drag solve (first order, L-stable). `hybrid` switches between them globally with hysteresis on two stiffness measures,

$$\zeta = \max_p \frac{\Delta t}{t_{s,p}}, \qquad \chi = \max \frac{\Delta t\, \rho_d}{\rho_g\, t_s}, \tag{1}$$

entering split-BE when either exceeds `hybrid_enter_{zeta,chi}` (0.5) and returning to PC2 only when both fall below `hybrid_exit_{zeta,chi}` (0.25); `hybrid_force_mode = pc2 | split_be` pins a branch, and `coupling = pc2` is that pin made permanent. The choice is reduced across ranks so every rank takes the same branch, and a `DUST_HYBRID_SUMMARY` line reports the cycle counts and the last ζ, χ. In the task graph (`dust/dust_tasks.cpp`) this appears as per-stage gates on every exchange and two migration chains; the deposited $Q, \mathbf{P}$ field gains a fifth feedback-rate channel; a second timestep task follows the split-BE migration.

Consequence for the program. The science targets (St = 0.1–10 at $\Delta t \sim 10^{-2}\,\Omega_0^{-1}$) sit in the resolved regime $\zeta \ll 1$, so `coupling = pc2` under `rk2` is the natural production setting: second order, about twice the speed of the IMEX dc1 path by the fork's measurement, no `imex2+` CFL penalty, and the same integrator as all gravity validation. Note that `hybrid` with the three-species NSH test ($t_{s,\min} = 0.01$) already trips $\zeta = 0.55$ and runs split-BE throughout; the validation document records the first-order signature that produces. The IMEX path stays as the reference for stiff configurations.

4.6 Build notes

`imex2+` needed nothing: it is upstream #612 and was already in the tree; the dust fork only refactored its coefficients into `SetImEx2PlusCoefficients`. Dust sources are part of every build (`src/CMakeLists.txt`); the four dust problem generators are built-in (`-D PROBLEM=built_in_pgens`, selected by `<problem>/pgen_name`). The rehearsal clone was built without network by linking the tree's `kokkos` checkout and pointing `FETCHCONTENT_SOURCE_DIR_KOKKOS-FFT` at the cached `build/_deps/kokkos-fft-src`.

5 Phase 4b (i): the shear-periodic deposit fold

5.1 The problem

A particle near a MeshBlock boundary deposits part of its weight into ghost cells of its own block; the additive exchange (§4.4) adds those ghost deposits into the active cells of the neighbor that owns the location. Across a shear-periodic x_1 face the owner's cells sit at a *different* y: the tree wraps `shear_periodic` exactly like `periodic` (`mesh/meshblock_tree.cpp`), so the x_1 buffers of the face blocks carry the deposits to the unshifted y — the "exact only for azimuthally uniform states" caveat the module used to print. The copy exchanges of hydro and of u^* fix this by *overwriting* the x_1 ghosts with a remapped copy after the plain exchange; an additive exchange cannot overwrite, so the unsheared contribution has to be suppressed and replaced.

5.2 The sign

The shearing-box identification used throughout the tree is $u(x, y) = u(x + L_x, y - q\Omega_0 t)$: the copy exchange fills the inner-x_1 ghosts with the outer boundary's content shifted by $+y_{\rm sh}$ ($y_{\rm sh} = q\Omega_0 L_x t$), and the outer ghosts by $-y_{\rm sh}$ (`multigrid/multigrid_shear.cpp:144-146`, `gravity/fft_gravity.cpp:414-416`). A *deposit* in an inner ghost cell at y is a contribution to the outer boundary at $y - y_{\rm sh}$; the deposit map is the adjoint (transpose) of the ghost-fill map, so its signs are reversed:

Adjoint signs. Inner-ghost deposits are shifted by $-y_{\rm sh}$ and added to the *outer* face blocks; outer-ghost deposits by $+y_{\rm sh}$ into the *inner* face blocks. Copying the multigrid fill's per-face signs verbatim gives the wrong answer.

5.3 The fold

`MeshBoundaryValuesDep::FoldShearDeposit(a, yshear, rcon)` (`bvals/bvals_dep.cpp`) reuses the Phase-1 global-plane machinery of `Multigrid::FillShearGhostPack` with three changes: the gather reads *ghost* slabs and is additive, the signs are the adjoint ones, and the scatter *adds* into active cells.

1. **Gather.** Two planes `shear_plane_(face, v*ng+d, gk, gj)` of the global y–z extent (uniform grid: `mesh_indcs.nx2`×`nx3`), zeroed, then every face block adds its whole x_1 ghost slab — all k, j *including the y/z corner ghosts* — at the periodically wrapped global index (face 0 ← inner ghosts `is-1-d`, face 1 ← outer ghosts `ie+1+d`). The corner rows overlap the active rows of the y/z neighbors' slabs, so the gather is an atomic *sum*: unlike the multigrid fill, several blocks legitimately write the same plane cell. Rows beyond a non-periodic z face are dropped (the vertical-boundary policy of §13 item 7; the dust guards currently require periodic z).
2. **MPI seam.** `Kokkos::fence` and one `MPI_Allreduce(SUM)` of each plane, reached by every rank in lockstep (the fold is a task; all ranks run the same graph, and every send a rank waits for is posted before the collective).
3. **Remap.** Per (face, variable, depth, z-row): the integer part of the shift is folded into a periodically wrapped scratch load, the fractional part `eps = fmod(ysh, dy)/dy` goes through `DC/PLM/PPMX_RemapFlx` (`shearing_box/remap_fluxes.hpp`) and the conservative update $q_j \leftarrow q_j - (F_{j+1} - F_j)$; $dy = L_y/$`gny` is the global rank-consistent cell width (the `ConsistentDx2` lesson). Face 0 uses $-y_{\rm sh}$, face 1 $+y_{\rm sh}$.
4. **Scatter-add.** Face 0 into the outer face blocks' `ie-d`, face 1 into the inner blocks' `is+d`, active k, j only.

The remap preserves the row sum, so the total deposited mass is conserved to round-off; the summation order in the gather and the reduction can differ with the rank count at round-off level only.

5.4 Suppressing the unsheared contribution

`RecvAndSumDeposit` skips, on a block whose inner (outer) x_1 face is shear-periodic, every receive buffer whose direction has $o_{x1} < 0$ (> 0): faces, x_1x_2 and x_3x_1 edges, and corners — all of which carry x_1-ghost content of the neighbor across the face. The direction of buffer index n comes from a 56-entry table built in the constructor from the slot layout documented in `mesh/nghbr_index.hpp` (faces 0–7, x_1x_2 edges 16–23, x_3x_1 edges 32–39, corners 48–55) and cross-checked against `NeighborIndex` at construction.

A pitfall worth recording. The first version built that table by calling `NeighborIndex` over all direction and sub-index arguments. For edges the function uses only the first sub-index, and the second landed on *other* slots — the x_3x_1 edges of one sign overwrote those of the other. The plain pass then still delivered the unsheared x_3x_1-corner deposits, which the unit test caught as an off-by-one in the multiplicity check (§5.6) and as a 10^{-4} drift of the 3D NSH equilibrium.

5.5 Hooks and parameters

Two tasks, `DustGasDrag::FoldDepQP` and `FoldPMBR` (`dust/dust_tasks.cpp`), follow `RecvDepQP` and `RecvPMBR` with the same per-coupling stage gates; the implicit solve now depends on the fold. `CompleteAddExchange` (coupled solvers) calls the fold as well. The offset is `DustGasDrag::ShearOffset()` $= q\Omega_0 L_x\, t$ at `pmesh->time` on every stage, the convention of the u^* remap. `<dust>/shear_remap` selects the order (`plm` default, as for u^*; `ppmx` available), `<dust>/shear_fold = false` is a diagnostic that skips the fold (the face blocks' x_1 deposits are then dropped, not unsheared). The warning the module printed under shear is replaced by an information line.

5.6 The test generator

`pgen_name = dust_deposit_shear` (`pgen/tests/dust_deposit_shear.cpp`, `inputs/dust/dust_deposit_shear.athinput`) exercises the exchange without particles: the x_1 ghost slabs of the face blocks are filled with known data, every other cell is zero, and the exchange runs exactly as the module runs it. Ghost deposits are *partial* sums (a block's corner ghosts overlap its neighbors' active rows), which the first version of the test got wrong by filling every ghost cell with the full function; the corrected test carries three fields: a smooth $g(x, y, z)$ periodic in y, z on the slab rows inside the block's own y/z range (one source per location, so the strips must equal g at the shifted location up to the remap error), a constant on the same rows (exact for any shift), and a constant on the *whole* slab whose received value must equal the analytic overlap multiplicity $(1 + n_y)(1 + n_z)$ of the source row (exact at integer shifts). The generator also reports the largest value leaked into any active cell outside the receiving strips (must be exactly zero) and the relative mass change (round-off). `problem/fold = false` runs the plain pass alone.

5.7 A structured, rank-independent initial condition

The NSH generator's random placement hashes each particle's tag to a block index *within the local pack*, so the realization depends on the rank count (as does `dust_damping`'s, seeded by the pack's first GID) — fine for physics runs, useless for serial-vs-MPI comparisons; a divergence that this produced was first mistaken for a bug in the fold (the no-back-reaction control agreed to 10^{-13} because test particles never feed positions back). `dust_nsh` therefore gained `<problem>/mass_mod_amp`, `mass_mod_phase`: on the lattice, particle masses are multiplied by $1 + A\sin(2\pi(y - y_{\min})/L_y + \phi)$, a deterministic azimuthally structured dust distribution (with `check_equilibrium = false`). `inputs/dust/dust_nsh_shear3d.athinput` is the tracked 3D shear-periodic NSH input with all Phase-4b keys present for command-line overrides.

6 Phase 4b (ii): the ideal-gas energy update

6.1 The problem

The dust fork applied the drag back-reaction to the gas momentum only and therefore required an isothermal EOS (a fatal guard). Gravito-turbulence needs cooling, hence an ideal gas, and then every change of the gas momentum must be accompanied by a change of the total energy: otherwise the internal energy $e = E - |\mathbf{M}|^2/2\rho$ silently absorbs the negative of the kinetic-energy change.

6.2 The rule

`GasKick(u0, m,k,j,i, dm, ideal)` (`dust/dust.hpp`) is the single entry point for a drag kick $\Delta\mathbf{M}$ on a cell: for an ideal gas it adds $\Delta E = \big(|\mathbf{M}+\Delta\mathbf{M}|^2 - |\mathbf{M}|^2\big)/2\rho$ before adding $\Delta\mathbf{M}$, so the kick leaves e unchanged. The four sites where the module changes the gas momentum all use it: the IMEX back-reaction apply (`ApplyPMBR`), the IMEX history term (`AddDragHistoryGas`, $\Delta\mathbf{M} = \tilde{a}_{22}\Delta t\,\mathbf{R}_g$), the PC2 predictor apply (`GasImplicitSolve`, stage 1) and the hybrid commit (`ApplyPMBR`, stage 2, with the tableau-weighted predictor removal folded into the same kick). The coupled solvers only read the gas state and apply through `ApplyPMBR`.

Frictional heating (`<dust>/drag_heating`, default `false`). The dissipation of one drag kick $\Delta\mathbf{v}_j = c_j(\tilde{\mathbf{u}}_j - \mathbf{v}_j)$ on particle j — the kinetic energy lost by particle and gas together beyond the gas kinetic-energy change that `GasKick` books — is

$$Q_j = m_j\, c_j\,(1 - c_j/2)\,|\tilde{\mathbf{u}}_j - \mathbf{v}_j|^2 \;\geq 0, \tag{2}$$

deposited with the particle's weights into a fourth channel of the back-reaction field `dmom` (now four variables; the exchange objects follow) and added to E where the momentum channels are applied; the IMEX history term scales it with the same $\tilde{a}_{22}$. On the PC2 path the midpoint impulse gives $Q_j = m_j[\Delta\mathbf{v}_{\rm drag}\cdot(\mathbf{u}_g - \mathbf{v}_h) - \frac{1}{2}|\Delta\mathbf{v}_{\rm drag}|^2]$, the midpoint quadrature of $m_j|\mathbf{u}-\mathbf{v}|^2/t_s$ over Δt in kinetic-energy form. With an isothermal EOS the flag is ignored with a warning.

6.3 What the bookkeeping cannot do, and the lesson from the first test

Inside a low-storage RK stage the conserved state is combined linearly, $\mathbf{u} \leftarrow \gamma_0\mathbf{u} + \gamma_1\mathbf{u}_1$; E combines linearly but $|\mathbf{M}|^2/2\rho$ does not, so the combined internal energy exceeds the combination of internal energies by $\gamma_0\gamma_1|\mathbf{M}_a - \mathbf{M}_b|^2/2\rho$ per step, where $\mathbf{M}_a - \mathbf{M}_b$ is the stage-1 momentum change — for a drag-dominated cell, the stage-1 kick. The default mode therefore does *not* keep e exactly fixed over a step; it gains this positive gap, first order in Δt (the kick scales with Δt while $\Delta t \ll t_s$) and sub-first-order while $\Delta t \gtrsim t_s$ of the stiffest species, where the kick saturates. The particle velocities are combined the same way, so the heating mode carries the analogous gap on the dust side: exact conservation of $E_{\rm gas} + K_{\rm dust}$ is not attainable in this framework, and the measured residuals (validation document §6) are this truncation error. Both modes converge with Δt; both are inert for an isothermal gas.

The first energy test looked like a 30% bookkeeping error; it was the history file's six significant digits (a total energy of 2.5 cannot show changes of 10^{-6}). `inputs/dust/dust_damping_ideal.athinput` therefore uses $p_{\rm gas} = 10^{-5}$, comparable to the dissipated energy, and `<time>/dtmax` to pin the step. The NSH generator gained the ideal-gas initialization it lacked (uniform `<problem>/pgas`); `inputs/dust/dust_nsh_shear3d_ideal.athinput` is the ideal-gas twin of the 3D input.

7 Phase 4b (iii): the particle timestep under shear

7.1 The problem

The inherited transport limit (§3.6) is $\Delta t \le c_{\rm p}\,\min_p \Delta x_2/|v_y - q\Omega_0 x|$ with $c_{\rm p} = $ `dt_cfl` $= 0.5$: one cell of the *total* azimuthal motion per step. The background shear is a smooth translation the drift integrates exactly ($y \leftarrow y + \Delta t\,(v_y - q\Omega_0 x)$ is linear in x), not a signal the mesh has to resolve; the gas, orbitally advected, is limited by its peculiar velocity only. In the $L_x = 1$ test boxes the two limits happen to coincide with the sound-speed limit, which is why the penalty never showed. In an $L_x = 8$ box with the NSH input's $\Delta x_2 = 1/32$ the particle limit is $0.5 \cdot (1/32)/(1.5 \cdot 4) = 2.6 \times 10^{-3}$ against a gas limit of 1.37×10^{-2}: a factor 5.3, growing with L_x.

7.2 What the limit is actually for

Two mechanisms depend on how far a particle moves in one stage, and neither is a cell:

1. **The deposit halo.** The TSC stencil reaches one cell beyond the particle, and `MeshBoundaryValuesDep` exchanges one ghost layer. Both are satisfied whenever the particle lies *inside its owning block* at deposit time — which is what the per-stage migration establishes: after `ParticlesBoundaryValues::SetNewPrtclGID` every particle carries the `gid` of the block that contains it. The halo-violation abort in `DepositDrag` (`dust/dust_pm.cpp`) is precisely the detector of a failed migration.
2. **The migration.** `SetNewPrtclGID` forms the integer offset of the particle from its block, $i_x = \lfloor (x - x_{\rm min} + L_x^{\rm mb})/L_x^{\rm mb} \rfloor - 1$ and likewise in y, z, and indexes the neighbor table with `NeighborIndex`$(i_x, i_y, i_z, \ldots)$. The table covers faces, edges and corners: offsets in $\{-1, 0, 1\}$. A particle that traverses a whole MeshBlock in one stage produces $|i_y| = 2$, an out-of-range neighbor slot, and a wrong or invalid destination. The one exception is a crossing of the shear-periodic x_1 mesh boundary, routed by the (y, z) shear maps of Phase 4a for any azimuthal offset.

So the hard requirement is *less than one MeshBlock per stage*, in every direction, on the total transport velocity; the cell-per-step limit on the peculiar velocity stays for accuracy, as for the gas.

7.3 The rule

`DustGasDrag::ComputeNewTimeStep` (`dust/dust_newdt.cpp`) now reduces two minima over the particles in one kernel and takes the smaller:

$$\Delta t_{\rm cell} = c_{\rm p}\,\min_p \min_i \frac{\Delta x_i}{|v_i^{\rm cell}|}, \qquad v_y^{\rm cell} = \begin{cases} v_y & \texttt{dt_transport = relative} \\ v_y - q\Omega_0 x & \texttt{full} \end{cases} \tag{3}$$

$$\Delta t_{\rm block} = \frac{s}{\beta_{\rm max}}\,\min_p \min_i \frac{L_i^{\rm mb}}{|v_i^{\rm full}|}, \qquad \beta_{\rm max} = \max_{\rm stages} |\beta_s|, \tag{4}$$

with $v_y^{\rm full} = v_y - q\Omega_0 x$ in the 3D shearing box (and $v^{\rm full} = v$ otherwise), $L_i^{\rm mb}$ the block extent, and $s =$ `<dust>/dt_block_safety` (default 0.5). The factor $\beta_{\rm max}$ is what makes s the literal fraction of a block crossed per stage. In a low-storage stage the position is combined as $y \leftarrow \gamma_0 y + \gamma_1 y_1 + \beta_s \Delta t\,\dot{y}$ with $\gamma_0 + \gamma_1 = 1$, so the displacement from the position the stage started with is $\gamma_1(y_1 - y) + \beta_s \Delta t\,\dot{y}$; for the tableaus in use the register term never adds to the drift term in magnitude, and the per-stage displacement is bounded by $\beta_{\rm max}\Delta t\,|\dot{y}|$:

integrator	stage displacements / $(\Delta t\,\lvert\dot{y}\rvert)$	$\beta_{\max}$	bound used
`imex2+`, $\gamma = 1 + 1/\sqrt{2}$	1.707, 0.707	1.707	1.707
`imex2+`, $\gamma = 1/2$ (switch)	0.5, 0.5	1.0	1.0
`rk2`	1.0, 0	1.0	1.0
`rk3`	1.0, 0.5, 0.5	1.0	1.0

The kernel reads `pdrive->beta[]` over `nexp_stages`, so the γ-switch is covered without a table. The premise $\gamma_0 + \gamma_1 = 1$ with non-negative weights holds for every tableau the module's integrator guard admits (`imex2+` for `coupling = imex`, `rk2` for `pc2/hybrid`) and for `rk3`; `rk4` has $\gamma_0 < 0$ and would need its own bound, but cannot be selected. Note the pre-existing asymmetry the cell limit carries under `imex2+`: $c_{\rm p} = 0.5$ is applied to Δt, while stage 1 advances $\gamma\Delta t = 1.71\Delta t$, i.e. 0.85 cells per stage; it is unchanged here and harmless (nothing in the module needs less than a block).

7.4 Parameters and diagnostics

`<dust>/dt_transport = relative|full` (default `relative`; `full` is the legacy limit, bit-identical to Phase 4b (ii)) and `<dust>/dt_block_safety` (default 0.5; a value ≥ 1 is accepted with a warning for the diagnostic run of validation §7 that shows the halo abort the guard prevents; the constructor prints which limit is in force in a 3D shearing box). The guard is always on, in all three directions and in every geometry; outside a shearing box it is inert in practice (the cell limit is stricter by $N_i^{\rm mb}$). The destructor prints `DUST_DT_SUMMARY block_guard_cycles=`n, the number of cycles in which the block guard was below the cell limit *on rank 0* — a rank-local count that says which of the two particle limits was the active one, not whether the particles limited the run at all (the gas usually does once the shear is out). The two 3D shear-periodic inputs carry both keys so they can be varied from the command line (§14).

7.5 What the larger step exposes

Once the shear is out of the limit, the step in a wide box is the gas step, and for species with t_s below it the IMEX coupling runs in its stiff regime. The validation scan (§7 there) shows this is a plain, converging truncation error — second order, 1% in the quiet phase and 26% during a streaming burst at $\Delta t/t_{s,\min} = 1.4$ — that the legacy limit had been masking in wide boxes only. The operating rule that follows is recorded in the validation document; it does not change the timestep code, which now limits what must be limited and nothing else.

7.6 What does not change

The push, the deposits, the fold and the migration are untouched: a larger step simply means that particles now hop MeshBlocks routinely in y (up to $s\,N_2^{\rm mb}$ cells per stage) and cross the shear-periodic face with multi-cell azimuthal offsets, both paths that existed and were exercised before at one cell per step. The split-BE branch of the hybrid coupling keeps its later timestep task (`NewTimeStep2`), which calls the same routine.

8 Phase 4c: dust self-gravity

8.1 Design, as built

The design of §2 is implemented without change of plan: one Poisson solve per stage on the *total* density, the gas feeling the total potential through its existing source term, the particles through a

mesh force gathered with the deposit kernel. The pieces, in the order they run each active stage (figure of §2.2):

1. **Deposit** (`DustGasDrag::DepositMass`, `dust/dust_gravity.cpp`), in the `before_stagen` list: $\rho_d = \sum_p m_p W(\mathbf{x}_p)/V$ with the module's PM kernel, at the pre-stage positions — the same time level as the gas density u_0 the solve reads. Particles sit inside their owning blocks (last stage's migration), so the stencil reaches at most one ghost layer.

2. **Additive exchange and fold**: `MeshBoundaryValuesDep` (one variable) sums the ghost deposits into the neighbours and `FoldShearDeposit` takes them across the shear-periodic faces exactly as for the drag deposits (§5).

3. **Ghost fill**: a copy exchange (`MeshBoundaryValuesCC`, one variable) with the shear remap (`ShearingBoxCC`) fills the ghost layers of ρ_d from the neighbours' active cells, because `Multigrid::LoadSource` loads the active zone *plus one ghost layer* of its source (`multigrid.cpp:290`), which for the gas is valid by the hydro boundary fill. The shear offset uses the module's `ShearOffset()` $= q\Omega_0 L_x t$, the same instant the multigrid freezes its own phase at (`mg_gravity.cpp:325`).

4. **Registration**: `Gravity::RegisterExtraDensity(`ρ_d`)` (once, in the dust constructor; `pgrav` is built before `pdust`). `Gravity::Solve` then forms $\rho_{\rm tot} = u_0(\texttt{IDN}) + \rho_d$ over every cell into its own array, and the three readers of §2.2 — `LoadSource`, the slab-plane gather (`ComputeSlabPlanes`, which gained an index argument) and the FFT `GatherDensity` — read `SourceArray()(m,SourceIndex(),k,j,i)`: the gas array at `IDN` when nothing is registered, the total at index 0 otherwise. Neither solver has any dust-specific code.

5. **The solve**, called by the driver between the two lists, unchanged — except that with the dust module present it is *skipped on the dormant imex2+ assembly stage* (`driver.cpp`): the module deposits nothing there, so the density would be stale, and nothing consumes ϕ ($\beta_3 = 0$). Gas-only runs keep their stage-3 solve, so their histories are unchanged.

6. **Force** (`ComputeForceField`), first in the `stagen` list: $\mathbf{g} = -\nabla\phi$ by centred differences on the active zone (ϕ is valid on the active zone plus one ghost layer for both solvers), with a zero-gradient fill of the ghost layers at physical (non-periodic) faces; then a copy exchange (three variables) and the shear remap, the pattern of the module's u^* field.

7. **Gather**, in `ExplicitPush`: the IMEX branch gathers $\mathbf{g}$ at the pre-stage position with the deposit kernel and adds it to the explicit force next to the Coriolis terms; the PC2 branch gathers it at the midpoint position with the weights it already has for u^* and adds $h\mathbf{g}$ to the right-hand side of its implicit-midpoint velocity solve, as it does the vertical tide.

The gas needs nothing new: `<hydro_srcterms>/self_gravity` applies $-\rho\nabla\phi$ with the same centred gradient. The drag back-reaction deposits only the drag kick, so there is no double count of gravity.

8.2 Momentum conservation, exact by construction

With one kernel W for scatter and gather, the total force on the dust is $\sum_p m_p \sum_c W_{pc}\mathbf{g}_c = \sum_c \rho_{d,c} V \mathbf{g}_c$, and the total on the gas is $\sum_c \rho_{g,c} V \mathbf{g}_c$; their sum is $\sum_c \rho_{{\rm tot},c}\, V\, \mathbf{g}_c$. In the periodic solve $\rho_{\rm tot} \propto L\phi$ with L the symmetric seven-point Laplacian and $\mathbf{g}$ the antisymmetric centred difference, so $\sum_i (L\phi)_i(\phi_{i+1} - \phi_{i-1}) = 0$ identically: the coupling exchanges momentum between gas and dust

without creating any, to the tolerance the solver reaches. The validation document measures 10^{-13} (§8 there) for NGP and TSC alike. This is the reason for insisting on one kernel and on the mesh gradient rather than a direct particle–particle or particle–potential interpolation.

8.3 Synchronous entry points

Problem generators that call the solve outside the time loop use `AssembleGravitySourceNow()` and `ComputeGravForceNow()`, which run the same deposit, exchanges, fold and force with spin-waited receives (the idiom of the coupled drag solvers' `Complete*Exchange`). The static twin tests are built on them.

8.4 Parameters, guards, boundaries

`<dust>/gravity` (default: true when a `<gravity>` block exists), `gravity_source` (the dust in the Poisson source) and `gravity_force` (the particles feel ϕ); test particles in a gas potential are `gravity_source = false`. Dust gravity needs a 3D mesh. The boundary guard of the module now admits physical (non-periodic) x_3 faces in 3D with a warning: ghost deposits beyond such a face have no receiver and are discarded, the density ghosts there are zero and the force ghosts zero-gradient, and a particle that leaves through the face is not routed — the deposit halo check aborts on it. The vertical policy (removal or reflection, and the mass account) remains the Phase-4d item it was.

8.5 What was found on the way

`Particles::dtnew` uninitialized. The upstream defect of the survey (§13, last item) bit: the constructor never sets `dtnew`, the dust generators each set it by hand, a new generator that did not saw `Mesh::NewTimeStep` read garbage before the first cycle, and the $2\times$ growth rule then held $\Delta t = 0$ forever. Fixed at the source: the constructor initializes it to the float maximum, and `Driver::Initialize` computes the dust transport limit from the initial state as it does for the fluids. The existing runs are bit-identical.

Multigrid geometry. The multigrid requires *cubic cells* ($\Delta x = \Delta y = \Delta z$: its smoother is the isotropic seven-point operator) and, with periodic directions, *more than one MeshBlock across each* (a block that is its own neighbour breaks the level-boundary exchange); the slab-open sheet test gives errors of 10^2–10^3 otherwise and 4×10^{-11} on $32 \times 32 \times 64$ cells in a $0.5 \times 0.5 \times 1$ box. The FFT solver has neither restriction. Both Phase-4c dynamic inputs are shaped accordingly; the requirement belongs in the multigrid document.

A numerical instability of the drag coupling. The 3D NSH lattice, exact to round-off for ten orbits at $32 \times 32 \times 8$ (Phase 4b), grows a round-off perturbation by $1.6\times$ per step at $32 \times 32 \times 32$ ($\Delta z = \Delta x$) and at $32 \times 32 \times 24$, but not at $32 \times 32 \times 16$ or $32 \times 32 \times 64$ (validation §8, table). The scan localizes it: it needs the drift ($\eta v_K \neq 0$), the TSC kernel (CIC and NGP are stable at the same step) and $c_s \Delta t / \Delta z \gtrsim 0.35$ evaluated with the *cycle* step — `imex2+` at `cfl_number` 0.35 is stable and 0.40 is not, `rk2`/PC2 at 0.30 is stable and 0.45 is not, so it is not the stage Courant number of the gas — and it does not need the stiff species. The growth rate ($\sim 35\,\Omega$) rules out the streaming instability, which is also absent with CIC. It is a property of the inherited coupling, not of the gravity, and Phase 4b never met it because every 3D box so far had $\Delta z \geq 2\Delta x$. The operating rule until it is understood: `cfl_number` ≤ 0.3 in boxes with $\Delta z \leq \Delta x$ (the gravity NSH input carries it); the mechanism — an aliasing of the three-cell deposit with the vertical sound crossing is the natural suspect — is a Phase-4d item.

9 Phase 4d: the instability identified, the vertical policy, restart, diagnostics

9.1 The instability of §8 is the hydro scheme's

Two diagnostics settle what the scan of §8 could not. First, the eigenmode: binary outputs of the unstable 32^3 dust run (validation §9) show the growing perturbation in the *gas* velocity and density at the grid Nyquist in all three directions, $(k_x, k_y, k_z) \approx (16, \pm 15, \pm 15)$ of 32 — a 3D checkerboard — while the NGP-binned dust density stays flat until particles cross cells. Re-read with that mode's frequency, $\omega = 2c_s\sqrt{\sum_i \Delta x_i^{-2}}$, every row of the scan falls on one line, $\omega\Delta t \gtrsim 1.2$–$1.3$, including the Δz dependence. Second, the decisive test: the same box with *no particles*, its gas velocities seeded with 10^{-10} white noise (`<problem>/gas_vpert` of the NSH generator), grows the same mode at `cfl` 0.45 and 0.40 under both `imex2+` and `rk2`, and damps it at 0.30 and at $n_z = 8$. A uniform state has no perturbation to grow, whatever the scheme's stability — which is why the gas-only NSH "held to round-off" and the dust run, whose deposit round-off seeds every mode, did not. TSC seeded it where CIC and NGP happened not to; the drift made the seed non-symmetric. The instability is the shearing-box hydro's (HLLE, PLM, orbital advection) 3D checkerboard Courant limit on cubic cells, and the operating rule `cfl_number` ≤ 0.3 in 3D is a hydro rule the SC14 gravito-turbulence runs already obey. The Phase-4c wording "TSC drag-coupling instability" is superseded.

9.2 Vertical policy: outflow removal

The Athena-C reference (`par_strat3d.c`, `zbc_out = 1`) treats the vertical faces as outflow for particles: a particle that crosses one is gone. Implemented in three parts. `ParticlesBoundaryValues::SetNewPrtclGID` (`bvals_part.cpp`) tests, before the neighbour lookup, whether a crossing particle has left the mesh through a *physical* face — neither periodic, shear-periodic nor internal, from `mesh_bcs` — and marks it dead with `PGID` $= -1$ instead of routing it. `Particles::RemoveDead` (`particles.cpp`) compacts the arrays (an exclusive scan over the survivors into fresh arrays) and refreshes the Mesh's per-rank and total particle counts; it is collective under MPI (an `Allgather`), so every rank calls it whenever a migration ran. The dust tasks `RemoveEscaped` / `RemoveEscaped2` follow the two migrations' receives in the task graph, and the drag deposit and the back-reaction commit now depend on them; they sum the removed mass first, and the destructor prints `DUST_ESCAPE_SUMMARY removed=`n `mass=`m (reduced over ranks). Ghost deposits beyond a physical face are discarded as before, and the density and force ghosts there are as in §8. With every face periodic the marking never fires and the existing runs are bit-identical.

9.3 Particle restart

The restart file gains two pieces. In the writer's serial header (step 3, after the turbulence RNG state) the root writes the per-rank particle counts and the two array widths; their $(n_{\mathrm{ranks}} + 2)$ integers are added to the header size the parallel offsets are computed from. After all MeshBlock data (a new step 5) each rank writes its particle arrays at an offset that follows the lower ranks' blocks (shared file) or its own MeshBlock data (one file per rank), every value as a `Real` in variable-major order — the integer properties are small, and a device-independent layout matters more than the bytes. The reader mirrors both: the counts reset the Mesh's bookkeeping and the array sizes before the MeshBlock reads (the constructor had sized them from `<particles>/ppc`), the arrays are read after them, and the problem generator is then called with `restart = true` as before. The dust generators already return early on restart after enrolling their history functions; the stopping-time contract is

re-checked at the first `GammaSwitch` from the restored `IPTS`.

9.4 Diagnostics

`dust_dpm` is a new output variable (slot 155): the dust density as the module deposits it — its kernel, the additive exchange and the shear fold — obtained from `AssembleDustDensityNow()` (the synchronous assembly of §8, now available in every dust run: `rho_dust` and its exchange objects are allocated unconditionally). `dust_d` remains the NGP binning of upstream. The SC14 history (`GravitoTurbHistory`) gains seven columns when a dust module exists: the dust mass, the three momenta (shear-relative velocities, as the particles carry them), the kinetic energy, the Reynolds stress $\sum m v_x v_y$ and the cumulative escaped mass; the gravitational-stress columns already measure the total potential. The NSH history's mass-weighted momenta are from Phase 4c.

The particle history (`file_type = phst`). The counterpart of the Athena-C dust code's `dump_particle_history.c` (`outputs/particle_history.cpp`): one line per output time in `<basename>.phst` with the global scalars — particle count, mass, the three momenta and kinetic energies (shear-relative velocities, as the particles carry them), the escaped mass, and the maximum of the module's own particle-mesh dust density (`AssembleDustDensityNow`, the Roche-density diagnostic) — followed, per species, by the count, the mean position and velocity, and the number-weighted standard deviations $\sigma_x, \sigma_y, \sigma_z, \sigma_{v_x}, \sigma_{v_y}, \sigma_{v_z}$ (Athena-C's "var" columns, $\sqrt{\sum (q - \langle q \rangle)^2/(n-1)}$; σ_z is the dust scale height H_d). Means and deviations are two exact passes with MPI sums, so the file is rank-independent to round-off. The stage-2 input writes it every 0.25.

The Table-1 script. `scripts/analysis/baehr_table1.py` forms their Table 1 from a run directory: Q, α_R and α_G from the gravito-turbulence history (their eqs. 21–22 from the volume-weighted `wrey`, `wgrv`, `rho_cs2` columns), H_d and the velocity dispersions from the `phst`, the diffusion coefficients from tag-matched displacements between `pvtk` snapshots (x wrapped over L_x, z unwrapped; y is sheared and unused), $Sc = (\alpha_R + \alpha_G)/\delta_{d,z}$, and, with `-roche`, a census of the cells above the Roche density $3.5\,\Omega^2/G$ in the `dust_dpm` dumps (connected components and their masses).

9.5 The dusty Jeans test

`ProblemGenerator::DustJeans` (`pgen/tests/dust_jeans.cpp`) with `scripts/analysis/dust_jeans.py`: the generator takes every component of the eigenmode as a parameter (gas density and velocity amplitude and phase, per species the dust density — carried by the particle masses — and velocity), the script solves the $(2N+2)$ linear system of Krapp et al. (2024, Eqs. 50–54) for the fastest-growing mode and prints the command-line overrides (including `four_pi_G` for $k_J = 1$, the mesh and box for the requested k/k_J and cells per wavelength — cubic cells, `-nx1`, `-ny` — the species and `tlim`), and fits the growth rate from the generator's history of mode projections. `-nspec N` builds Krapp's many-fluid mixture (N species sharing ϵ_0, t_s linear — or log-uniform with `-ts-log`, which is what their Fig. 6 actually shows — between `-ts-min` and `-ts-max`); since the command line cannot add keys absent from the input, `-input`/`-out` then writes a complete input file with the $5N$ species keys in place. Validation §8.

The rotating problem. The same generator in an unstratified shearing box (`inputs/dust/dust_jeans_rot.athinput`: shear-periodic x_1, a `<shearing_box>` block, $\Omega = 1$, $q = 3/2$): the axisymmetric $k_y = 0$ mode gains an azimuthal peculiar velocity in every component (`vgy_amp/phase`, `vdy_amp_s/phase_s`), which is all the generator adds — the Coriolis and shear terms, orbital advection and

the shear-periodic deposit fold are the modules' own. The script's `-omega` branch builds the $(3N+3)$ system (`matrix_rot`: velocities peculiar, $2\Omega v_y$ in x, $-(2-q)\Omega v_x$ in y, `four_pi_G` explicit) and `check` pins it to its closed forms: the gas-only $s^2 = 4\pi G\rho_0 - \kappa^2 - c_s^2 k^2$ (Chandrasekhar 1961), the frozen-gas cubic $s[(s+1/t_s)^2+\kappa^2] = 4\pi G\rho_d(s+1/t_s)$ whose strong-drag limit $4\pi G\rho_d t_s/(1+\kappa^2 t_s^2)$ is the secular instability of Ward (2000) and Youdin (2005, 2011) and is k-independent, $\Omega \to 0$ against the Krapp matrix, and $G \to 0$ epicycles — all to 10^{-14}. Two things the eigenproblem itself settled before any run: the selection of the fastest mode must tolerate ties in $\Re s$ (the $G \to 0$ oscillations carry a 10^{-10} drag damping that would otherwise lose to the zero mode), and the eigenvector must not be normalized by $\delta\rho_g$ when that component is negligible (the neutral mode at the back-reaction threshold is a pure dust pattern). `fit` now also reports $\Im s$ from the phase of the projection and, with `-auto`, restricts itself to the linear window (the longest span over which every column is a single exponential below a set amplitude): the slow modes of long runs are overtaken by other modes of the 3D box, notably the vertical dust Jeans mode, which rotation does not stabilize and which grows from round-off at $\sim 0.7\,\Omega$. Validation §8.

9.6 The science setup: Baehr, Zhu & Yang (2022) in two stages

The target runs (their Table 1, models `BR_L_10` and `noBR_L_10`) are built in SC14 code units with $H_g = c_{s0}/\Omega = 1.02 \times 2.12625 = 2.16878$ — $Q_0 = 1.02$ through c_{s0} — so their $512^2 \times 256$ box with $L_x = L_y = (80/\pi)H_g$, $L_z = (40/\pi)H_g$ is $55.23^2 \times 27.61$ at $\Delta = 0.1079$ (cubic, $0.05\,H_g$). Two additions serve it. `<hydro_srcterms>/bcool_cs2_floor` gives β-cooling an irradiation floor: only the internal energy above the floor temperature c_s^2 = `bcool_cs2_floor` is cooled (their background irradiation that holds Q above Q_0; zero recovers SC14). And the runs are staged: the gas alone to saturation (`inputs/shearing_box/gravito_turb_baehr.athinput`, $t = 50$, dumps every 10), then a restart that *inserts* the particles — `athena -r dump -i gravito_turb_baehr_dust.athinput`: the add-on's blocks merge over the dump's input, `<particles>/restart_insert = true` makes the restart reader skip the particle section the dump does not have, and `ProblemGenerator::GravitoTurbInsertDust` places the layer: `ppc` particles per cell in total (1.5×10^6), uniform in (x, y), Gaussian in z with width `dust_hz` $= H_g$ (their initial gas width), at rest in the shearing frame, one species at $t_s = 1/\Omega$ ($St = 1$), equal masses summing to $Z = 0.01$ of the gas mass in the box. Per-block counts follow the Gaussian mass in each block's z range and positions come from a hash of the global block id and the particle's index, so the insertion is decomposition-independent; the Mesh counts and tags are rebuilt. PC2 coupling under the gas run's `rk2`, `cfl` 0.3, TSC, self-gravity on, diode faces with the outflow removal. Back-reaction on or off is one command-line switch (`dust/back_reaction`). Continuing a stage-2 run from one of its own dumps needs `particles/restart_insert=false`, since those dumps carry the particles. Job scripts: `scripts/cluster/gt_baehr_stage{1,2}.slurm`, README §3.

Several sizes at once. The insertion places `<dust>/nspecies` species interleaved round-robin within every block (species s = particle index mod N), so each has the same count and the same spatial distribution; species s carries `<problem>/dust_Z_s` of the gas mass (default `dust_Z`$/N$) in equal-mass particles, the per-block counts rounded to a multiple of N. The add-on `gravito_turb_baehr_dust3.athinput` is the three-species layer $St = 0.01, 0.1, 1$ (`ppc` $= 3 \times 0.02235$: 1.5×10^6 particles per species, $Z = 0.01/3$ each) for the question of which sizes lock into the spirals and which are stirred; with back-reaction off the species do not interact, so one run is three single-species experiments. Two things came out of exercising it (validation §8). First, the coupling: the $St = 0.01$ particles that land in the floor-density halo make $\epsilon/t_s \sim 4 \times 10^4$ there, and the explicit gas-side drag of PC2 cannot take it — Δt collapses to 5×10^{-5}, the halo gas is blown up and a quarter of the dust leaves the box. The add-on therefore uses `coupling = hybrid`: PC2 while a cell's $\chi = \Delta t\,\epsilon/t_s$

is below 0.5, split backward-Euler drag where it is not (the merged module's own switch, `DUST_HYBRID_SUMMARY`); `coupling = imex` under `imex2+` is the other stiff-safe choice. (A third thing came out of the first production attempt, commit `95c3e955`: stage 2 died at cycle 1 on Vista with an invalid MPI request in `ParticlesBoundaryValues::ClearPrtclSend`, at 128, 512 and 2048 ranks alike and never on the laptop. The particle exchange's counters `nsends`, `nrecvs`, `nprtcl_send`, `nprtcl_recv` had no initializer — harmless under the stock particle task list, where a clear always follows a send — but the dust graph clears the exchange at the end of every active stage and the hybrid coupling migrates only at stage 2, so the first clear ran before any migration had happened and waited over a garbage count: zero by luck on the laptop, not on Vista. The counters are now initialized and the clears loop over the requests they hold.) Second, the optional radial pressure gradient: the constant acceleration on the gas of the NSH tests, $a_x = 2\Omega\eta v_K$ (`hydro_srcterms/const_accel`, keys present in the add-on, off by default), is balanced by the uniform azimuthal offset $v_y = -a_x/(2\Omega) = -\eta v_K$, so the insertion applies that offset to the restarted gas (`problem/gas_vy_shift = auto`; total energy updated); without it the switch-on launches the undamped box-wide epicycle of amplitude ηv_K. The stage-2 script takes `ADDON` and `OVR` from the environment.

The slab-plane cost, found on Vista. The first stage-1 submission ran at $\sim$ 25 s per cycle on 16 nodes — never finishing. The cause was in the multigrid's slab boundary (`MultigridDriver::ComputeSlabPlanes`): it gathered the *whole* root-resolution density on every rank ($512^2 \times 256$ doubles, 0.54 GB per rank), Allreduced it across the 2048 ranks, and then every rank redundantly rolled and transformed all 256 planes, twice per cycle. Tolerable at SC14's 3M cells, it made the cost grow with the box rather than with a rank's share of it (and was most of the "memory-bandwidth-bound" cycle cost measured earlier). It is rewritten (multigrid document, Phase 3b addendum): each rank transforms only the root k-planes its own blocks touch, the shear roll is an exact phase between the y and x transforms (linear, so the per-rank partial spectra add up; the earlier limited remap did not), and one Allreduce of the two face spectra (8 MB) replaces the 0.5 GB one. The static slab tests are unchanged to every digit, the sheared gravito-turbulence run matches its archive until the shear phase builds and differs at 10^{-6}–10^{-4} thereafter (the planes are now free of remap error), 4 ranks now reproduce serial exactly, and a 64^3 box went from 25 s to 0.33 s per cycle.

And the same code again, on a GPU. Batching the slab gather and forward transforms over a stack of planes (`73204c3a`, `bf1a4446`) removed the launch-latency floor that dominated the solve on a GPU and took the $16H$ box from 4.9×10^7 to 9.2×10^7 cell-updates per second. It also, for a week, made every stage-2 run above 64 ranks fail — of signal 9 at 128 ranks and of a UCX *Transport retry count exceeded* on an `RDMA_READ` at 512, which read as a fault of the interconnect and was not. The stack was sized by a compile-time batch of 32 and the y transform ran on a non-trailing axis, so kokkos-fft allocated and freed two arrays the size of the whole stack inside every call: 320 MiB held and 256 MiB churned per rank at the Baehr root grid, 72 GiB of slab buffers on a 128-rank node, and the churn is what invalidates the registration cache UCX serves RDMA from. Commit `cb3b4b8a` puts every transform on a trailing axis (the shear-phase kernel carries the index swap) and sizes the batch from a memory budget, 48 MiB per rank on a host build and 2 GiB on a device build; the box falls to 36 MiB per rank and the slab tests are unchanged to the last digit. The multigrid document has the full account, under "The slab planes on a GPU, and the memory fault they hid".

10 Phase 5: particles on statically refined meshes

Branch `dust-smr`, off `dust-multigrid` at `fbf52141`. Static mesh refinement (SMR) only: adaptive refinement moves, creates and destroys MeshBlocks and nothing in the particle module follows (`PGID` would go stale, and no hook redistributes particles), so the dust constructor is fatal on `adaptive`. Within SMR the shearing-box policies of the gravity track apply unchanged: both shear-periodic x_1 faces at one uniform level, refined regions spanning the full x_2 extent under orbital advection, and at most one level of refinement for the dust (lifted in Phase 6, §12.5; the reason is in §10.2). Gravity-free tests first; the gravity path is wired but belongs to the next validation step.

10.1 What the dust module relies on, and what refinement breaks

Four things changed meaning. (1) The additive deposit exchange (`MeshBoundaryValuesDep`) only knew same-level neighbors: ghost cells of one block overlap active cells of the other one for one. Across a level jump a fine ghost cell covers a quarter (2D) or an eighth of a coarse cell, and a coarse ghost cell covers 2^d fine cells. (2) The copy exchanges of the module's own fields — the provisional gas velocity u^*, the dust density and the gravitational force — passed a one-element dummy as the "coarse array" that `PackAndSendCC` reads for a coarser neighbor, and nothing prolongated their fine ghosts. (3) The particle migration of upstream (`SetNewPrtclGID`) handles a finer neighbor by the particle's sub-block and a coarser one by walking the sub-block slots until one is set — which is wrong at the *interior* edges and corners of a coarse face, where `MeshBlock::SetNeighbors` deliberately sets no slot (that region belongs to the block that also owns the face), so the walk ran into an unrelated slot or off the table. (4) The shear-periodic machinery of the particles (the (side, l_{x3}, l_{x2}) $\to$ gid maps) and of the fold (the global y–z planes) were laid out on the root grid.

10.2 The deposit kernel across a level jump

The first design was the obvious conservative one: a fine block restricts its ghost deposits (the volume average of the 2^d fine ghost cells covering each coarse cell) and a coarse block's ghost deposits are injected as the same density into the 2^d fine cells they cover. Both directions conserve the deposited mass exactly, and the deposit unit test and the structured rank-independence tests pass. But the uniform NSH lattice does not hold: the deposited density is not uniform at the interface, because a particle's TSC footprint has half the reach on the fine side. Consider a planar interface with the coarse cell C_0 adjacent to it and fine cells F_0, F_1 beyond, all particles on a lattice at their own cell centres with masses $\propto$ cell volume. C_0 receives $3/4 + 1/8$ from its own side, but from the fine side only the restriction of what the fine kernel put into the fine ghost cells, $\frac{1}{2}(1/8 + 0) = 1/16$ instead of the $1/8$ a coarse-lattice neighbor would have given: $15/16$. F_0 receives $7/8$ from its side and $1/8$ by injection: 1. F_1 receives its full 1 and the same injected $1/8$: $9/8$. Mass is conserved ($-\frac{1}{16}V_c + \frac{1}{8}V_f = 0$) but the field carries an $O(1)$ two-cell ripple at every interface, independent of resolution — in expectation over random particle positions as well, with the same $15/16$ and $9/8$ (the tails $\int_{1/2}^{\infty}$ of the CIC and TSC B-splines are both $1/8$). The dust_dpm output of the SMR lattice showed exactly 1.125 in the second fine cell (validation §10). The NSH equilibrium drifted at the 10^{-5} level per orbit.

No local rule repairs this while keeping per-particle conservation: depositing into the coarse neighbor with the *coarse* kernel makes the field uniform in expectation but $\sum_k W_{jk} \neq 1$ per particle (momentum conservation of the back-reaction lost); the restrict/inject rule conserves but is inconsistent. The scheme adopted does both:

Finest-level deposits. Every MeshBlock scatters with the kernel of the finest level. A block one level below the finest deposits into a 2× fine image of itself ($2n_x$ cells and $2n_g$ ghost cells per direction, cell volume $V/2^d$; `dust::DepositStencil`), which `DustGasDrag::RestrictImage` volume-averages onto the block's own cells, ghosts included, before the exchange. A block at the finest level deposits into its own cells as before. Gathers keep using the block's own cells.

A fine cell then receives the finest-kernel weight of every nearby particle whatever block it lives in, and a coarse cell the volume average of those weights: one consistent field, each particle's weights summing to one (the image is a partition of unity and the average preserves it), so the mass, the momentum antisymmetry of the back-reaction and the exactness of the closed-form drag solve are as on a uniform mesh. The price is the memory of the images (five, four and one variables at 2^d the block size, allocated only when the pack has a block below the finest level) and a consistency that holds across *one* level jump only: a block two levels down would deposit with the kernel of the level above it, restricted, which differs from its finer neighbor's; hence the guard. With the lattice placed at the finest-level cell centres on every block (the NSH generator's `lattice_finest`, 2^d particles per coarse cell with masses scaled to the finest volume) the deposited field is exactly uniform across the interface (1.000000000000000 in every cell) and the equilibrium is held to round-off again; with one lattice point per cell of each block the two spacings' kernel sums differ by $O(1)$ at the interface (a lattice property: for random positions both are uniform in expectation), so that test is documented as a property of the lattice.

Superseded in storage, not in result: the image and `RestrictImage` were replaced on October 19, 2026 by folding the fine kernel onto the block's cells (§12.6); the deposited field is the same to round-off, and every statement of this subsection about it still holds.

10.3 The additive exchange across levels

`MeshBoundaryValuesDep` gains the level-dependent index sets of the cell-centered exchange, built in its own `InitSendIndices`/`InitRecvIndices` to mirror `MeshBlock::SetNeighbors`:

- *Fine sender, coarser receiver* (`icoar` send, `ifine` receive). The sender packs its ghost shell restricted to coarse resolution (`RestrictDeposit` averages the deposit array, ghost shell included, into an internal coarse copy), $n_g/2$ coarse cells deep, over its full transverse extent *plus* an overhang of $n_g/2$ coarse cells on the interior side of its parent (the side toward its sibling): the neighbor table stores a coarser neighbor at an edge or corner only when it is an *exterior* edge of the coarse face, so the interior overhang travels with the face buffer, exactly the adjoint of the copy exchange's $(cn_x - n_g)$ overhang. The coarse receiver sums it into $n_g/2$ active cells of its half of the face extended by $n_g/2$ cells across the midline.
- *Coarse sender, finer receiver* (`ifine` send, `icoar` receive). The sender packs, for each of its n_g ghost cells over the quadrant (f_1, f_2) of that fine neighbor, the 2^d cells of its fine image (the finest-kernel deposits of its own particles); the payload is 2^d times the cell count. The fine receiver adds each into the fine cell it covers, $2n_g$ fine cells deep. (Since §12.6 these values come from the block's 2× halo image.)
- Same level: unchanged, bit for bit.

The rank-packed messaging sizes each payload from these index sets (`GetVarDataSize`), so sender and receiver agree by construction; the receiver's scalar loop over neighbor buffers (the overlapping strips) is kept. The SMR variant needs $2n_g \le n_x$ in each active direction (fatal otherwise). The synchronous and coupled-solver paths pass the same fine image; the coupled Schur-complement solvers are fatal under refinement (their exchange helpers were not extended).

10.4 Copy exchanges, prolongation, and the shear machinery

u^*, the dust density and the force each get a real coarse array (the layout of `Hydro::coarse_u0`). `RestrictField` (`MeshRefinement::RestrictCC`) runs before each `PackAndSendCC`, and a new task after each receive chain — `ProlongateUstar` before the gather, `ProlongateRhoDust` before the Poisson solve, `ProlongateGravForce` before the push — does what `Hydro::Prolongate` does: `FillCoarseInBndryCC`, a zero-gradient fill of the coarse ghosts at physical faces (so the prolongation stencil of a block touching both a coarser neighbor and a diode face reads defined data), then `ProlongateCC`. All no-ops on a uniform mesh.

The particle shear maps and the fold's planes are built at the (uniform) level of the shear-face blocks, found from any block with $l_{x1} = 0$; `FindTargetMB`'s convention. Interior refined blocks never touch a shear face, so the fold itself is unchanged; the deposit unit test with the two face columns refined to level 1 reproduces the uniform test at that geometry to every digit.

10.5 Particle migration

`DestinationSlot` (`bvals_part.cpp`) replaces the slot walk. For an offset (i_x, i_y, i_z) it takes the finer neighbor's sub-block from the particle's half of the block, the same-level slot 0, or the one sub-block slot a coarser neighbor occupies; if none is set — an interior edge or corner of a coarser neighbor — it drops the offset components in which the block is interior to its parent (its parity within the parent, recovered from the block's position on the mesh) and retries with the face or edge that remains, at most twice. A remaining miss aborts with the particle's coordinates instead of routing it to a wrong block. On a uniform mesh the result is the old one.

10.6 Files, hooks, parameters

Generic (cherry-pickable to the dust fork): `bvals/bvals_dep.cpp` and `bvals.hpp` (level index sets, `RestrictDeposit`, fine-image packing, planes at the face level), `bvals/bvals_part.cpp` (`DestinationSlot`), `bvals/bvals.cpp` (shear maps at the face level), `dust/dust_smr.cpp` (`ZeroImage`, `RestrictImage`, the latter removed in §12.6), `dust::DepositStencil` in `dust.hpp` and its use at the six scatter sites (`DepositDrag` twice, `GatherKickPMBR`, the PC2 advance, the split-BE preparation, `DepositMass`), `RestrictField`/`ProlongateField` and the three prolongation tasks in `dust_tasks.cpp`, the coarse arrays and guards in `dust.cpp`; the test generators `dust_settle` (new) and `dust_nsh` (`lattice_finest`), `dust_deposit_shear` (planes at the face level); inputs `inputs/dust/{dust_nsh_shear3d_smr, dust_deposit_shear_smr, dust_deposit_shear_smr_faces, dust_settle, dust_settle_smr, streaming_linA_smr}.athinput`. Gravity-side: the rho_dust and force prolongation hooks in `dust_gravity.cpp`. No new `<dust>` parameter: refinement is read from `<mesh_refinement>`; the module prints one line when it is active.

11 Gravity on statically refined meshes

Phase 5 wired the gravity path through the refined mesh and left it untested. The wiring is small, because the gravity fields ride the machinery the drag already needed: $\rho_{\rm dust}$ is deposited with the finest-level kernel like every other deposit field, its copy exchange is restricted and prolongated like u^* (`SendRhoDustCopy`, `ProlongateRhoDust`), and the force is restricted, exchanged and prolongated the same way (`SendGravForce`, `ProlongateGravForce`), all in `dust_gravity.cpp` with `coarse_rd` and `coarse_g` allocated in `dust.cpp`. Exercising it (validation document §11) required two fixes and turned up one limitation that belongs to the gravity solver.

11.1 `MeshRefinement` now precedes the `ProblemGenerator`

`main.cpp` step 5 built `Mesh::pmr` after the generator, with the comment that its buffer sizes depend on the physics modules. They do, and `Mesh::AddCoordinatesAndPhysics` has already run at that point; nothing in the constructor reads the initial conditions. But a generator that restricts or prolongates while setting them up — the dust twin does, through `AssembleDustDensityNow`, which must hand the multigrid a $\rho_{\rm dust}$ whose ghost layer is filled across levels — then dereferences a null pointer: `DustGasDrag::RestrictField` calls `pmesh->pmr->RestrictCC` and every such run died with `SIGSEGV` inside `MeshRefinement::RestrictCC`. The object is now constructed immediately after `AddCoordinatesAndPhysics` and before either `ProblemGenerator` branch. No code uses a null `pmr` as a flag for "not refined" (`Mesh::multilevel` is that flag), and the change moves no arithmetic: every number on every mesh is unchanged.

11.2 Volume weighting of the `fft_poisson` diagnostics

The generator's reductions run over the active cells of every MeshBlock on the mesh but normalised by `mesh_indcs.nx1*nx2*nx3`, the root-grid cell count. On a refined mesh those differ by the refinement volume ratio, and the quantity that suffers is not a norm but the physics: the mean density subtracted from the triply-periodic right-hand side. At 1.875 (a 64^3 root grid, central octant refined) the reference run reported a relative residual of $0.6366 = 1.75/2.75$, uniform over every cell, while the solver's own volume-weighted `Multigrid::CalculateAverage` had done the right thing and its defect norm was 2.6×10^{-7}.

Each summand now carries `LevelWeight`$(l, l_{\rm root}, d) = 2^{-d(l-l_{\rm root})}$, the cell's volume relative to a root cell, and each mean and rms is normalised by `wcells_tot`, their sum ("root-cell equivalents"). The weight is a power of two, so the weighting is exact; on an unrefined mesh every weight is exactly 1.0 and `wcells_tot` is exactly the root cell count, which keeps every uniform number bit for bit as it was. All five reduction groups are weighted: the convergence study (§8), the residual and TS23 §4.1 error, the shearing-wave and sheet comparisons, and the MG-versus-FFT difference. The same normalisation issue exists in `dust_nsh`'s `DustNSHErrors`, whose gas mean is $\sum \rho v / \sum \rho$ over cells and whose dust mean is per particle: both are exact for the uniform states the equilibrium test uses (and for `lattice_finest`, where every particle has the same mass), so they were left alone, but they would be biased for a structured state on a refined mesh.

11.3 New diagnostics

- `# TS41-MESH-FORCE` forms $-\nabla\phi$ on the mesh with the same centred difference the gas source term (`srcterms.cpp`, as $\frac{1}{2}(\delta\phi_l + \delta\phi_r)/\Delta x$) and the dust gather both use, against the analytic gradient of the triple-sine potential: max, volume-weighted rms, max on the finest level, max on coarser levels, and the volume fraction of cells above 1% of the peak force. It needs no particles, which is what makes it able to say whether a force defect is the solver's or the dust module's.
- `# DUST-GRAVITY FORCE ERROR` gained the same five fields over the particles, and `# DUST-GRAVITY FORCE ARGMAX` reports the position, block level and in-block cell index of the worst particle.
- `<problem>/probe_line` prints ϕ, $\phi_{\rm analytic}$, $-\partial\phi/\partial x_1$ and its analytic value for every cell of the x_1 row through the worst cell, ghost layers included, which is how the interface boundary layer of §11.4 was measured. Off by default.

11.4 The coarse-fine boundary layer of the potential

The measured result (validation document §11.4) is that the converged multigrid potential on a statically refined mesh is second-order accurate in the L_2 sense — better at a given root resolution than the uniform mesh — and that its *gradient* within a few cells of a coarse-fine interface is not accurate at all: the maximum relative force error is 9×10^{-2} and stays there as the mesh is refined ($9.53, 9.03, 8.74 \times 10^{-2}$ at root $32^3, 64^3, 128^3$), while the uniform mesh converges at second order ($3.17 \times 10^{-3}, 8.00 \times 10^{-4}, 2.01 \times 10^{-4}$). It is not an iteration artifact: driving the defect norm to 7×10^{-10} leaves the number identical to seven digits, and V-cycles alone, four ghost zones and an unlimited prolongation limiter all reproduce it.

The mechanism is that the interface values the solver leaves behind are consistent with the operator it inverted, not pointwise samples of ϕ. The discrete Laplacian residual is at round-off in the interface cells, yet ϕ there carries an error that grows geometrically toward the boundary — about 7×10^{-6} in the interior of a $\Delta x = 1/128$ block, 3.5×10^{-5} three cells out, 2.9×10^{-4} in the last active cell, 1.05×10^{-3} in the ghost the coarser level filled. A gradient taken across one cell divides such an error by $2\Delta x$ and turns a few times the truncation level into an $O(1)$ fraction of the peak force. Nothing local repairs it: one-sided second-order differences read the same degraded active cells, and the ghost is not a value one may difference. The repair has to be in the solver's own interface treatment (`MultigridBoundaryValues::ProlongateFCMG` and its restriction counterpart), which is shared by gas and dust and is not a dust-track file, so it is recorded here and left.

Two properties limit the damage and are worth stating precisely, because they are what make the science box usable at all. First, gas and dust take the *same* discrete force — the gas source term differences ϕ, the dust gathers $g = -\nabla\phi$ built from the same ϕ with the same stencil — so the pair conserves momentum exactly even where both are wrong: the structured self-gravitating state at rest holds its total momentum at 10^{-16} on the refined mesh. Second, the interface reproduces constants exactly, so a uniform density gives no force at all and the NSH lattice with gravity holds its equilibrium to round-off across level boundaries. What a level boundary does cost is measurable in orbital terms: test particles falling through a refined midplane twenty times over five periods lose 0.64% of their amplitude where the uniform runs that bracket the same resolution lose 0.06–0.09%, and their period error is worse than either, so refinement across a trajectory buys negative accuracy. They also spread over (x, y) by 3×10^{-4}, eight times the uniform run at the same finest spacing (3.9×10^{-5}; the coarse uniform run holds them to 2.5×10^{-8}, so part of that growth is the multigrid block decomposition rather than refinement), because the discrete force acquires a transverse structure that the physical potential does not have.

Operating rule for now. Do not place a coarse-fine boundary where the self-gravitating dynamics is delicate: keep the refined region large enough that its boundary sits in gas whose self-gravity does not matter, and prefer refinement that follows the density rather than cutting through the layer being studied. Momentum and uniform states are safe; a resolved gradient across a level plane is not.

11.5 Files, inputs, parameters

Generic: `main.cpp` (the `MeshRefinement` ordering), `pgen/tests/fft_poisson.cpp` (`LevelWeight` and the weighting of every reduction, `# TS41-MESH-FORCE`, the force-error split and argmax, `<problem>/probe_line`). Inputs `inputs/dust/{dust_poisson_twin_smr, dust_nsh_shear3d_grav_smr, dust_grav_orbit_smr}.athinput`, and `<problem>/probe_line` added to the two twin inputs so the command line can set it. Analysis: `scripts/analysis/grav_orbit_period.py` (the period by quadrature at an arbitrary release amplitude, the amplitude drift and the (x, y) spread; it generalises

the Phase-4c `orbit_period.py` that hard-coded $z_0 = 0.03$). No change to `dust_gravity.cpp` or to any other dust file: the gravity path as Phase 5 built it is what these tests certify.

12 Phase 6: particles on adaptively refined meshes

Phase 5 left one sentence in the dust constructor: adaptive refinement is fatal because nothing moves the particles when MeshBlocks are created, destroyed or handed to another rank. The gas side of the shearing box already had full AMR — the ring and x_1-face policies, the ring-wise synchronisation of the refinement flags, the rebuild of the boundary GID lists after every remesh — so this phase adds the particle side and nothing else: the particles follow the tree, the dust module rebuilds the little per-block state it keeps, a criterion on the particle-mesh dust density drives the refinement, and the sampling problem the linA diagnosis exposed (§10) is answered by splitting the particles of a block that refines; and, for the proposal figure that motivated this phase, the finest-kernel deposits are generalised to any number of levels (§12.5). The `drag_solver = local` requirement stands.

12.1 Redistribution inside the remesh

`MeshRefinement::RedistAndRefineMeshBlocks` builds the new tree, the old-to-new GID map and the new load balance before it touches any data, and only at its end switches the `Mesh` and the pack to the new numbering. The particles are routed in between, while the pack still carries the *old* numbering (its `gids`, `mb_size`) and the per-old-block refinement flags of that step hold $+1$ (refined), -2^d (derefined) or 0: `Particles::RedistributeAfterRemesh` (`particles/particles_amr.cpp`) is called right after the flags are set. It hands `ParticlesBoundaryValues::RemapAfterRemesh` three device tables — the new GID of each old block of this pack (its own, its first child's, or its parent's), the action, and the rank of every new GID — and one kernel assigns every particle its new GID: the old block's map entry, plus, when the block was refined, the child index $f_x + 2f_y + 4f_z$ from the particle's half of the block in each direction, which is the tree's child order (`MeshBlockTree::Refine`: child n has offsets $n\,\&\,1$, $(n \gg 1)\,\&\,1$, $(n \gg 2)\,\&\,1$). Particles whose new block lives on another rank are appended to the same send list `SetNewPrtclGID` uses (the append through `UpdateGID`, the count and validation through the new `FinishSendList`), and `ExchangeNow` runs the ordinary particle pipeline — count, post receives, pack and send, receive and unpack, clear — to completion outside the task list. Positions are untouched; the particle keeps its tag. The Mesh particle counts are refreshed by the collective `RemoveDead` at the end (nothing is dead; the Allgather is what is wanted).

After the new MeshBlocks and neighbour tables exist, the routing maps of the shear-periodic faces are rebuilt (`ParticlesBoundaryValues::InitShearMaps`, the constructor's map build factored out: AMR renumbers the face blocks and may move them), and `AdaptiveMeshRefinement` calls `DustGasDrag::ReinitAfterMeshUpdate` next to the hydro's `ShearingBox::ReinitAfterMeshUpdate`: the per-block factors `rfac/any_coarse` (now `rfac/halo`, §12.6) of the finest-kernel deposits from the new levels (`SetRefinementFactors`, shared with the constructor), the per-block global (x_2, x_3) offsets of the deposit fold planes in each `MeshBoundaryValuesDep` (`InitShearOffsets`, likewise factored out), and the shear remaps of u^*, $\rho_{\rm dust}$ and g. Every field and buffer array was already sized for `max_nmb_per_rank`; the fine images were allocated grow-only once some block was coarser than the finest level present (and, since §12.6, the halo images once some block has a finer neighbour).

12.2 Splitting on refinement

A coarse-sampled population entering a finer level keeps its coarse sampling: a lattice at twice the fine spacing is aliased by the fine kernel into an $O(1)$ comb, a random population's shot

noise rises by 2^d (validation document, §10.5). With `<particles>/split_on_refine = true`, `Particles::SplitOnRefine` runs before the remap: every particle of a refining block becomes 2^d children at the parent position offset by $\pm\Delta x_{\rm old}/4$ in each direction — half a fine cell, so a lattice at the old cell centres becomes exactly the lattice at the new cell centres — each with the parent's velocity, drag rates and stopping time, the RK position registers shifted with the position, and $1/2^d$ of the mass. The parent's slot becomes child 0; the others are appended with fresh tags above the global maximum (one `Allreduce` and one `Allgather` fix the per-rank ranges). Mass and momentum are conserved exactly and the sampling per cell is preserved on the fine level. What splitting cannot do is keep a *lattice* exact at the edge of the refined region: the children of a lattice form another lattice (the corners of the fine cells, for a lattice at the fine centres), and where two different lattices meet the truncated partial sums of the kernel leave the interface cells at 7/8 and 9/8 of the uniform density, the Phase-5 artifact in a new guise (validation document, §12.2). A random population has no such term in expectation, so the science runs split and the lattice tests do not.

The first science runs exposed the obvious hazard of splitting without merging: a block whose maximum sits near the threshold refines, derefines at the next interval, refines again, and each refinement quadrupled its particles (the AB run went from 1.05 to 1.44 million particles in one time unit with twelve blocks refined; the three-level BA run from 0.26 to 0.76 million in ten). Two remedies, both kept. Each dust particle now carries a *sampling level* (`PLEV`, a fourth integer: the level of its block when it was created, or read from an older restart file — set by `Particles::InitSamplingLevel` from the `ProblemGenerator` constructors *after* the generator has returned, because the generators assign the block id after creating the tags: the first version initialised it inside `CreateParticleTags` and left every rank but the first with garbage, validation document §12.6), children take the new level, and a particle is a parent only if its sampling level is below the new level: a derefined block that refines again keeps its particle count, and a particle that migrates into a block refining beyond its own level is still split. Restart files written before the fourth integer existed are read with the level set to that of the particle's block. And the criteria gained hysteresis: `<amr_criterion>/value_max_deref` (default `value_max`) derefines only below a second, lower value, and the generator's density mode leaves a level only once the maximum has fallen below `amr_hyst` (default 0.5) of that level's threshold. A child of a parent within half a fine cell of the block edge lands just outside its block: the deposit halo tolerates one fine cell, and the next migration pass routes it to the neighbour. Nothing merges particles on derefinement; a derefined block simply carries more particles. The routing itself runs whether or not splitting is on.

12.3 The dust density criterion

`<amr_criterion>/variable = dust_rho` (`method = min_max`, `value_max` to refine a block whose maximum exceeds it, derefining below) is the particle-mesh dust density of the module's own kernel, `rho_dust`, a shallow slice in `RefinementCriteria`. It is only up to date when something deposits it, so on the check cycles (`ncycle_check`) `CheckForRefinement` calls `DustGasDrag::AssembleDustDensityNow` — the synchronous deposit with the additive exchange, the shear fold, the copy exchange, the shear remap and the prolongation, the same chain the `dust_dpm` output and the Poisson source use — when any criterion names it. The cost is one deposit and its exchanges per check cycle.

12.4 The 2D r–z shearing box

The two refinement policies of the shearing box exist for machinery that runs only in 3D (or the unimplemented 2D r–ϕ): the shear-periodic remap and orbital advection tasks are guarded by `three_d || shearing_box_r_phi` in the hydro task list, and the 2D r–z box has plain periodic x_1 faces.

Phase 5 nevertheless applied both policies there (the linA input refined two full-height radial columns). They are now gated to `three_d` in `MeshRefinement` and in `Mesh::CheckShearingBoxRefinement`, so the 2D box refines in patches; nothing changes in 3D.

12.5 Any number of levels

Phase 5 allowed one level of dust refinement because a block deposits through a $2\times$ image and the exchange injected image cells straight into a finer neighbour's cells, which is consistent only if that neighbour is at the finest level. Both statements generalise. A block l levels below the finest now scatters into a 2^l image (`rfac(m)` $= 2^{l_{\max}-l}$, images allocated at the root block's factor; `DepositStencil` took r all along) and `RestrictImage` averages r^d image cells onto each of its cells. In the exchange, the message to a finer neighbour still carries 2^d values per ghost cell — the cells at the *neighbour's* level, which is one finer than the sender's by the 2:1 rule — and each value is now the mean of the $(r/2)^d$ image cells it covers (`PackAndSendDeposit`, the per-block factors handed over by `SetImageFactors`; without them the old $2\times$ reading is kept, which is what the deposit unit test uses). Messages to coarser neighbours restrict the block's own cells, as before, and same-level messages send them. So every cell of every level receives exactly the finest-kernel weight of every nearby particle, averaged over its own volume, whatever the levels of the blocks around it; the constructor's one-level fatal is gone. The generator's `lattice_finest` places $2^{d(l_{\max}-l)}$ points per cell accordingly.

12.6 Image-free deposits

Why. The 2^l image made the deposit memory and work of a coarse block grow as 8^l: with 8^3 MeshBlocks and $n_g = 4$ an image is $(16r)^3$ cells, ten variables over the three images, i.e. 2.6, 21, 168 and 1340 MB per block for $r = 2, 4, 8, 16$, allocated on *every* block of a pack with a coarse block and zeroed and restricted in every deposit. On the $(0.2H)^3$ stratified box (the `box02` runs) this is what stopped the adaptive mesh: three levels above the $320/H$ root did not fit on a 754 GB Turin node and needed four, and the four-level smoke test was killed out of memory as its finest level came in.

The fold. The restriction of an image is linear and acts on each particle's weights separately, so it can be done per particle before anything is stored. In one direction, a particle's finest-kernel stencil is three weights w_{-1}, w_0, w_1 on the fine cells $f_p - 1, f_p, f_p + 1$ (fine index space of the image, ghosts included). The image cell f lies in the block cell $\lfloor f/r \rfloor$, and averaging r^d image cells of volume V/r^d onto a cell of volume V sums their weights, so the block cell c receives

$$W_c = \sum_{n=-1}^{1} w_n \, \delta_{c,\lfloor (f_p+n)/r \rfloor}, \tag{5}$$

with the deposit divided by V instead of V/r^d. Three consecutive fine cells span at most two block cells for $r \geq 2$, so W is again a three-point stencil, about $\lfloor f_p/r \rfloor$, and the product over directions is the separable 27-point stencil of the existing scatter loops. `dust::BinWeights` does this fold; `DepositStencil` now takes the factor s of the target grid ($s = 1$: the block's cells, $s = 2$: half cells) and folds by $g = r/s$ (no-op when $g = 1$, so a block at the finest level, and every block of a uniform mesh, is arithmetically unchanged). The result equals the old image + `RestrictImage` to round-off: a one-dimensional check over 2×10^4 random positions for each $r = 2, 4, 8, 16$ agrees to 2.2×10^{-16}.

The halo image. One consumer still needs sub-cell resolution: the message to a *finer* neighbour carries the 2^d half cells of each ghost cell (the cells at that neighbour's level). A block with a

finer neighbour (`halo(m)` $= 1$, which implies `rfac` ≥ 2) therefore also folds the deposits of its *edge* particles — those whose block-cell stencil reaches a ghost cell (`StencilAtEdge`) — onto a $2\times$ halo image of itself ($s = 2$, $2(n_x + 2n_g)$ cells per direction). Interior particles never contribute to a ghost cell, so they skip the second fold. The halo image has the layout of the old one-level image, its size is independent of r (2.6 MB per 8^3 block with $n_g = 4$), it exists only when some block of the pack has a finer neighbour, and `ZeroImage` clears it only on those blocks. `PackAndSendDeposit` reads it at the fixed factor 2; `SetImageFactors` and the per-block factors of the exchange are gone, and so is `RestrictImage`.

At the six scatter sites (`DepositDrag` twice, `GatherKickPMBR`, the PC2 advance, the split-BE preparation, `DepositMass`) the scatter loop became a local lambda taking the target array, the stencil and the cell volume; it runs once into the block's array and, for an edge particle of a halo block, once more into the halo image with the $s = 2$ stencil (the back-reaction sites at the same pre-drift or midpoint position as before). The expressions inside are unchanged, so uniform runs are bitwise identical.

Checks. Each against the unmodified code (`9aaa5639`) on the same inputs (validation document, section "Image-free deposits"): uniform runs bitwise; the two-level SMR NSH equilibrium (`dust_nsh_shear3d_smr3`, IMEX, PC2 and hybrid couplings), the gravity SMR equilibrium, the Poisson twin (NGP and TSC) and the shear-periodic deposit exchange with refined interior or faces identical to the printed digits; the AMR sphere collapse with two and three levels bitwise (every dusty block is at the finest level there); a mixed SMR sphere with particles on root and level-1 blocks next to finer ones identical except the net momenta and centre of mass, which differ at 10^{-18}. The two-level AMR sphere ran in 191 s instead of 317 s.

What remains. On the `box02` branch (one Turin node, 256 ranks) three levels now fit: 577 GB right after setup, 617 GB at the peak, against 747 GB and an out-of-memory kill before; about 4.3 s per cycle on one node against about 4.7 s on four. The base of ≈ 2.25 GB per rank is the preallocation for `max_nmb_per_rank` $= 256$, not the dust. The cost per cycle is now set by the particle load: every MeshBlock has cost 1 in `LoadBalance`, and 2^3 splitting per level puts most particles on the few ranks that own the clumps. Four levels ran at about 30 s per cycle and were killed (signal 9, the node at 700 of 754 GB, probably at a remesh). The three-level run ended at cycle $\approx 156\,990$ in the negative-gas-density crash of the earlier runs (the `nan_check` repro tripped at cycle 156 984), which this change does not touch.

12.7 Files, hooks, parameters

Generic: `particles/particles_amr.cpp` (new: `RedistributeAfterRemesh`, `SplitOnRefine`), `particles/particles.{hpp,cpp}` (`split_on_refine`), `bvals/bvals_part.cpp` (`RemapAfterRemesh`, `ExchangeNow`, `FinishSendList`), `bvals/bvals.{hpp,cpp}` (`InitShearMaps`), `bvals/bvals_dep.cpp` (`InitShearOffsets`, `ReinitAfterMeshUpdate`), `mesh/mesh_refinement.cpp` (the two particle hooks in `RedistAndRefineMeshBlocks`, the dust hook in `AdaptiveMeshRefinement`, the `dust_rho` assembly in `CheckForRefinement`, the 3D gate), `mesh/refinement_criteria.cpp` (`dust_rho`), `mesh/build_tree.cpp` and `mesh/mesh.cpp` (the 3D gate). Dust: `dust/dust.{hpp,cpp}` (`SetRefinementFactors`, `ReinitAfterMeshUpdate`, the fatal removed). Generators and inputs: `dust_nsh <problem>/amr_test` with `amr_x1min/x1max`, `amr_x3min/x3max`, `amr_t_on/t_off` (a user criterion that refines a box for a time window, so a lattice passes through one refine and one derefine event), `inputs/dust/dust_nsh_shear3d_amr.athinput`; the Johansen & Youdin (2007) nonlinear runs

`inputs/dust/streaming_{AB,BA}_amr.athinput` (2D r–z, `dust_nsh` with random placement, AMR on `dust_rho`, splitting on; uniform twins by `mesh_refinement/refinement=none`). Figures: `validation/figures/plot_dust6_si_amr.jl`.

13 What the next phases must add

The survey established these gaps; they are listed in the order they bite.

1. **Shear-periodic additive deposit remap (4b). Done**, §5. The Phase-4c gravity deposit will be a third additive field and inherits the fold through `MeshBoundaryValuesDep`. The alternative design (image particles deposited directly in the neighbor's frame) stays in reserve; the field remap is exact for constants and second/third order otherwise, and it costs one small collective per fold.

2. **Ideal-gas back-reaction (4b). Done**, §6.

3. **Particle timestep under shear (4b). Done**, §7: limit on v_y, with a MeshBlock guard on the total velocity.

4. **Dust self-gravity (4c). Done**, §8, both solvers. The instability it exposed is the hydro scheme's 3D checkerboard mode (§9); `cfl` ≤ 0.3 in 3D.

5. **Particle restart (4d). Done**, §9.

6. **Diagnostics (4d). Done** in part, §9: `dust_dpm` and the SC14 dust history columns. Still open: `pvtk` omits stopping time and mass; a Roche-density / Hill-radius clump finder does not exist.

7. **Vertical boundary policy (4c/4d). Done**, §9: outflow removal with the mass accounted.

8. **Refinement (Phases 5 and 6). Done** for static refinement, §10: deposits with the finest-level kernel everywhere, the additive exchange across one level jump, prolongated copy exchanges, the migration fix, the shear machinery at the face level; and **done** for adaptive refinement, §12: the particles are routed to their new MeshBlocks inside the remesh, across ranks, the dust state is rebuilt, a criterion on the dust density drives the refinement, and particles are split on refinement (the answer to the linA diagnosis of the validation document, §10.5: a coarse-sampled population entering a finer level keeps its coarse sampling), with any number of levels (§12.5). Still open: the coupled drag solvers under refinement, merging on derefinement, and a load balance that counts particles (the cost estimate is one per MeshBlock). `<mesh_refinement>/prolong_limiter = none` (unlimited central slopes in `ProlongCC`) is available as a diagnostic.

9. **Gravity on refined meshes. Tested**, §11: the dust source is exact on every level (the NGP twin reproduces the gas-only reference on the same refined mesh to every digit, on 4 ranks too), the momentum antisymmetry survives at round-off, and the NSH lattice with gravity holds its equilibrium across level boundaries to round-off. Still open, and *not* the dust module's: the converged multigrid potential leaves a boundary layer at every coarse-fine interface whose gradient does not converge (§11.4), so neither gas nor dust feels the right force within a few cells of a level jump. The fix belongs in `MultigridBoundaryValues::ProlongateFCMG` and its restriction counterpart, and would want its own test ladder (a gas-only sound wave and a Jeans mode through an interface) before anything in the dust track depends on it.

10. **Upstream defects.** `Particles::dtnew` was never assigned by the constructor; **fixed** in Phase 4c (§8). `Mesh::pmr` was constructed after the `ProblemGenerator`, so any generator that restricts or prolongates while setting the initial conditions crashed on a refined mesh; **fixed** in §11.

14 Input-file interface

Required: `<time>/integrator = imex2+` for `coupling = imex`, `rk2` for `pc2` or `hybrid`; `<particles>/particle_type = dust`, `pusher = imex_dust`, `ppc` (particles per cell, an integer multiple of `nspecies` for lattice placement); `<mesh>/nghost >= 2`; periodic boundaries, or shear-periodic x_1 in 3D. Any EOS (the isothermal requirement for back-reaction is gone, §6). Shearing-box runs read `<shearing_box>/qshear`, `omega0`, `stratified`.

<dust> parameter	default	meaning
coupling	imex	imex (needs imex2+) / hybrid / pc2 (both need rk2); see §4.5
shear_remap	plm	dc / plm / ppmx: y-remap order of the shear-periodic deposit fold (§5)
shear_fold	true	false: diagnostic, skip the fold
drag_heating	false	ideal gas: deposit the frictional dissipation of each drag kick as heat (§6); default keeps e fixed and discards it
hybrid_force_mode	auto	auto / pc2 / split_be (hybrid only)
hybrid_enter_zeta, hybrid_enter_chi	0.5, 0.5	split-BE entry thresholds on ζ, χ
hybrid_exit_zeta, hybrid_exit_chi	0.25, 0.25	PC2 re-entry thresholds
nspecies	1	number of species; taus_1..N (positive) required
dust_to_gas	0.01	total dust-to-gas ratio for the default mass initializer
back_reaction	true	deposit the drag reaction onto the gas
deposit	tsc	ngp / cic / tsc assignment and gather kernel
dt_cfl	0.5	particle transport CFL factor (cell-crossing limit)
dt_transport	relative	relative: the cell limit uses the peculiar v_y; full: the legacy $v_y - q\Omega_0 x$ (§7)
dt_block_safety	0.5	fraction of a MeshBlock a particle may cross per stage on the total velocity (≥ 1: guard off, warns; diagnostic)
gravity	(<gravity> present)	dust takes part in self-gravity (3D only)
gravity_source	true	the PM dust density joins the Poisson source
gravity_force	true	particles feel $-\nabla\phi$ of the total potential
Refinement (<mesh_refinement>/refinement = static or adaptive, any number of levels) is supported with MeshBlocks of ≥ 2 nghost cells per direction and drag_solver = local (§10, §12); with adaptive, <particles>/split_on_refine (default false) splits the particles of a refining MeshBlock into 2^d children (once per level, by the particle's sampling level), <amr_criterion>/variable = dust_rho refines on the particle-mesh dust density, and value_max_deref adds hysteresis.		
gamma_switch	false	Krapp et al. eq. 18: $\gamma \to 1/2$ when $\Delta t > \max t_s$
stopping_time_mode	species_fixed	or particle_static, dynamic
drag_solver	local	local / dc1 / dc2 / pcg / adaptive / applya (coupled Schur-complement solvers; local is the BLKP19 closed form and the validated baseline)
drag_rtol, drag_atol, drag_iter_max	10^{-11}, 10^{-14}, 200	PCG tolerances
drag_adaptive_*	—	error target of the adaptive solver
drag_diagnostic_interval	0	DUST_SOLVER_DIAG print cadence

Problem-generator parameters (rho0, etavk, eps_s, placement, seeds, eigenmode amplitudes; dust_nsh's mass_mod_amp, mass_mod_phase, check_equilibrium; dust_deposit_shear's time0, remap, fold) and dust_nsh's and streaming_linear's lattice_finest (§10), dust_settle's z0, etavk, xsplit) are specific to the built-in generators dust_damping, dust_nsh, dusty_wave, streaming_linear, dust_deposit_shear, dust_grav_orbit, dust_settle; the files under inputs/dust/ and ../athenak_dust_tests/inputs/ are the reference configurations. Outputs: file_type = pvtk with variable = prtcl_all (positions, gid, tag, species, velocities); variable = dust_d on any mesh output (NGP mass density, needs a <dust> block); <time>/dtmax caps the step for fixed-step convergence tests.

Commit	Phase	Content
449b8f09	4a	Merge of dust/dust (d6de1296) into multigrid (b4d4c547); eight conflicts resolved keep-both; output slot grav_phi 153 → 154
a7c0def3	4a	Dust task graph: Prolongate before ApplyPhysicalBCs
774c6519	4a	bvals_dep.cpp: additive deposit exchange on the rank-packed MPI vars path (multi-rank deadlock fix)
78320aee	4a	Second merge, dust/dust at 1f3e1ed9: the fork's own rank-packed deposit exchange (taken), PC2/hybrid coupling, PM optimizations; three conflicts (§4.5)
6c0cafef	4b (i)	MeshBoundaryValuesDep::FoldShearDeposit, the x_1-buffer skip, the dust hooks, the dust_deposit_shear test generator, the 3D shear-periodic NSH input, the NSH mass modulation
489c656c	4b (ii)	GasKick, the four kick sites, the heat channel of the back-reaction deposit, <dust>/drag_heating, the isothermal guard removed, ideal-gas initialization of dust_nsh
375643e6	4b (iii)	DustGasDrag::ComputeNewTimeStep: cell limit on the peculiar velocity, MeshBlock guard on the total velocity; <dust>/dt_transport, dt_block_safety; DUST_DT_SUMMARY
9a96f9f3	4c	dust/dust_gravity.cpp; Gravity::RegisterExtraDensity and the source read of both solvers; the dormant-stage solve skip; the force gather in ExplicitPush; Particles::dtnew initialized; the twin and orbit generators, three inputs, NSH momentum history columns
d39858ac	4d	escape marking in SetNewPrtclGID, Particles::RemoveDead, the RemoveEscaped tasks; particle restart (writer step 3/5, reader); dust_dpm output; SC14 history dust columns; the seeded gas-only diagnostic (<problem>/gas_vpert); dust_grav_orbit vz0
a9279dc8	science	the Baehr et al. two-stage setup: bcool_cs2_floor, restart_-insert, GravitoTurbInsertDust, the two inputs, the Vista scripts
386d5968	science	the particle history output (phst) and scripts/analysis/baehr_table1.py
538fb43b	4c	the dusty Jeans test: ProblemGenerator::DustJeans, scripts/analysis/dust_jeans.py, the input and the figure
1506c83f	science	the multi-species insertion, the three-species add-on and the pressure-gradient equilibrium
f54271ba	4c	the rotating dusty Jeans test: the shearing-box branch of the generator and the script, the input and the figure
04dbd3dd	5	bvals/bvals_dep.cpp level index sets and fine-image packing, DestinationSlot in bvals_part.cpp, particle shear maps at the face level, dust/dust_smr.cpp (ZeroImage, RestrictImage), dust::DepositStencil at the six scatter sites, RestrictField/ProlongateField and the three prolongation tasks, the coarse arrays and guards; dust_settle, lattice_finest, six inputs
af45c756	5	streaming_linear lattice_finest and <mesh_-refinement>/prolong_limiter (the linA aliasing diagnosis)
c3d7b308	gravity/SMR	MeshRefinement before the ProblemGenerator in main.cpp; volume weighting (LevelWeight) of every fft_poisson reduction; # TS41-MESH-FORCE, the force-error split and argmax, <problem>/probe_line; three refined inputs and scripts/analysis/grav_orbit_period.py
1c2c0558	6	particles/particles_amr.cpp (RedistributeAfterRemesh, SplitOnRefine), RemapAfterRemesh/ExchangeNow/InitShearMaps in bvals_-part.cpp/bvals.cpp, InitShearOffsets in bvals_dep.cpp, the particle and dust hooks in mesh_refinement.cpp,